

报告正文 (2026 版)

(请勿删除或改动下述提纲标题及括号中的文字)

(一) 主要学术成绩、创新点及其科学意义 (建议不超过 5000 字)

着重阐述近 5 年来在基础研究方面所取得的学术成绩、创新点、研究价值和科学意义等。

1. 主要学术成绩

1.1 科学背景和学术成绩概述

一个大模型训练系统软件，负责调度一个大模型来充分使用加速卡集群的算力资源，对训练数据集不断地学习，从而不断提升该模型的智能。过去 10 年来，文本大模型在多个领域均取得了重大成就；驱动其智能迅速进步的关键，就是其参数规模与其训练数据集规模的迅速增长。因此，一个大模型训练过程所需计算和显存资源远超单卡，训练系统通常需要把一个大模型切分至大量切片，以某种并行维度的部署形式，分布式地部署到成千上万张加速卡中（例如，流水线并行和张量并行）。同时，训练系统须把训练数据集切分成多个子集，并把这些子集，以数据并行的维度，分别部署到大量加速卡中，并频繁地把大模型的众多子切片对各个数据集子集的训练增量进行跨卡同步。

每个并行训练维度对集群的算力、显存和跨卡通信等硬件资源的调度和使用，均存在特殊的科学问题。一个集成了这些并行维度的训练系统，常常面临一个更复杂的科学问题：如何更好地整合多个并行训练维度以及调度这些资源，从而最大化该训练系统在训练一个大模型时的有效总算力利用率（即训练系统的性能）。过去 5 年，国内外著名大学和企业，纷纷聚焦大模型多维并行训练系统的研究和产业化。但是，由于该科学问题的高度复杂性，国际已有相关系统仍面临很多开放性的重大科学难题，亟待解决。例如，英伟达在世界上大力推广的 Megatron 系统 [1] 和 Meta 最新于 2025 年发表的 FlexiblePP 系统 [2]，均正在面临数据并行维度通信，严重阻塞了流水线并行算力的难题。在我国典型的加速卡集群常常被多方共享共用的场景下，此难题常常会使得这些训练系统的有效总算力利用率降至大约 30% [3]。

我从清华大学计算机系获得本科和硕士学位，并于 2015 年从纽约哥伦比亚大学获得博士学位。我旋即加入香港大学，担任助理教授、博士生导师，从事并行和分布式训练系统的研究和产业化，主持了 13 项科研项目。针对文本大模型多维并行训练领域的科学问题以及系统软件实现，我主持了 1 项国家科技专

项（科技创新 2030-“新一代人工智能国家科技重大专项”重大项目）的支撑项目“大模型四维并行分布式训练系统”（项目代号为 2022ZD0160200，直接总经费 1.1 亿元人民币）。目前该支撑项目已完成结题报告，其中包含了我和博士生们针对该科学问题，取得的以下三个系列科研成果。

首先，是“训得快” (§1.2.1)。我发表 Fold3D 论文及其 AIAO 调度方法 (All-In-All-Out，即把前向和反向传播进行“全进全出”的重叠并行调度 [3])。比起国际的最先进相关方法，即英伟大 Megatron 系统的“交织 1F1B”调度方法，AIAO 使得“数据并行跨卡通信任务阻塞流水线并行计算任务”的数量，理论上大幅减少约 $(N - 1)/N$ ，其中 N 与模型参数规模以及训练该模型的加速卡数量相关。图 1 直观展示了两者的差异：交织 1F1B 往往把数据并行同步压到迭代尾部，而 AIAO 则把同步拆入按段推进的时间线，使更多通信被相邻计算覆盖。

Fold3D 已在支持国产华为升腾卡的 MindSpore AI 软件库中完成产业转化 [4]。该库中开源了五种大模型训练方法，其中暂时只有 Fold3D (即 FoldPipeline-Transformer) 是国产自研方法，其四种方法来源于美国论文（例如英伟达和斯坦福合研的 Megatron 方法 [1]，斯坦福自研的 Pipedream 方法 [5]，以及谷歌 GPipe 方法 [6])。2024 年在深圳 3072 张升腾 910B 卡上训练一个盘古大模型时，Fold3D 比 Megatron 性能显著提升约 30%。华为公司目前在全世界宣传和推广的大模型多维并行训练“计算和通信重叠并行”（或“计算和通信折叠隐藏”）这一科技标签，即来源于 AIAO 方法。

首先，是“训得稳” (§1.2.2)。大模型训练的其中一个核心并行维度，流水线并行，由于训练的“前向传播，再反向传播”，以及某些大模型的模型结构训练时动态变化等科学特性，常常使流水线并行的各阶段（各加速卡）的显存使用等硬件的负载及其不均衡，使流水线吞吐量、即训练的性能不稳、骤降的科学难题（例如，据我们实验，斯坦福 Pipedream 流水线方法在某些大模型动态训练负载下，各阶段算力利用率仅约 20%）。我发表 vPipe 论文 [7]，在国际上率先给出了流水线并行动态训练的负载均衡的近似最优解，及其动态调度方法。

在系统软件实现和技术上，vPipe 论文中提出“选择性重新计算”方法，实现了一系列典型大模型的流水线并行训练时高达约 90% 的均衡负载（包含各阶段的有效算力和显存使用量）。该策略已在支持英伟达加速卡的 MindSpeed AI 开源库中完成产业转化。在该开源库的三种支持流水并行的均衡显存重计算方法中 [8]，目前暂时只有 vPipe 的“选择性重新计算”方法是国产自研并且被该库标记为“推荐使用”；其余两种方法来源于美国（包含 Gpipe 的全量重算方法，以及

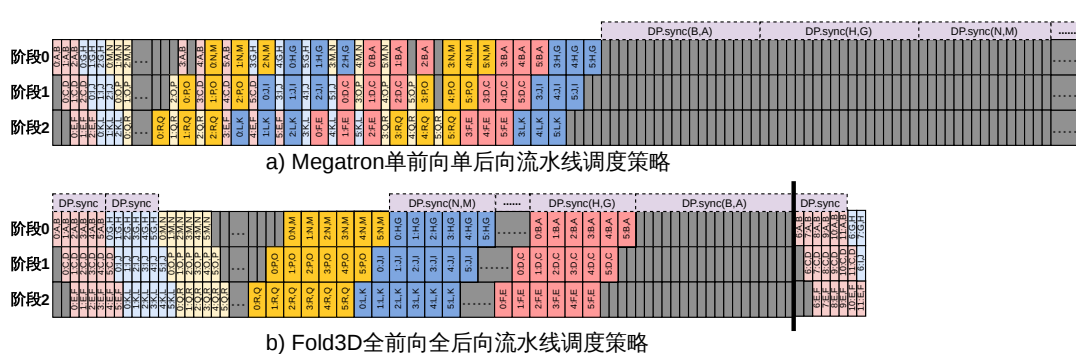
Megatron 的激活函数重算方法)。

第三，是“训得齐” (§1.2.3)。以上两项工作主要聚焦大模型的预训练。随着完成预训练的大模型种类越来越多，增强训练对每个大模型适配更专业、更特殊的应用场景，变得越来越重要。而使增强训练高效的一个重要趋势，是支持越来越长的输入序列（例如，常见的开源文本大模型一般是用长达 64K 甚至 128K 的文本输入序列进行的增强训练）。大模型增强训练的一个重要科学问题，是已有相关训练系统一般对大模型理解长文本序列的核心是注意力机制（attention）和混合专家机制（mixture of experts），而这两机制会产生很重负载、以及互相依赖的计算任务和通信任务。此依赖很容易导致这些任务的序列化执行（不并行、不对齐），使得增强训练过程各个加速卡的严重无效空转和低算力利用率。

我发表 FoldMOE 论文，在世界上率先提出“词元级”流水线并行新概念：把长文本序列按词元（token）进行巧妙切分，从科学上解绑了混合专家机制的全对全通信（all-to-all）对于整个注意力机制的依赖问题。我们并提出“1A1M 归一”、流水线并行式的调度新方法，从理论上几乎使所有计算和通信任务得以并行化和基本对齐，从而极大缓解了此类增强训练过程的加速卡无效空转。FoldMOE 已完成产业转化，并同时应用于大模型的增强训练和推理场景。例如，FoldMoE 已应用于国产著名大模型 DeepSeek-V3 的高性能并行推理：我们在加州伯克利推理平台 vLLM 上实现了“1A1M 归一”调度方法和国际最相关的微软 Tutel 调度方法 [9]，我们的方法比 Tutel 显著提升了 1.5 $\times$ ）。

尤其是，到 2025 年，随着我主持的国重点项目推进和 Fold3D 论文成果带动的软硬件国产化，华为发出一封表扬信（见简历）透露：（1）Fold3D 软件促进了千卡万卡集群在全国的销售落地和大范围推广，（2）Fold3D 是华为性能最好的大模型训练系统软件。环顾全球，在千卡万卡硬件集群上的所有单体系统软件及其应用场景中，大模型训练是少数具有显著经济效益的场景之一。因此，在过去 4 年，我的 Fold3D 工作帮助我国大模型训练软硬件的研发和部署达成了“三个数量级”的提升（从 2022 年十余张卡进行多维并行训练仍然困难，到 2025 年可以稳定运行千卡/万卡进行大模型超大规模训练）。这项提升有效降低了我国在这一关键科技竞争方向上被“卡脖子”的风险（见 §1.2.1 和 §1.2.2）。

从国际顶尖论文的发表数量上看，在我担任香港大学博士生导师的过去 10 年来，仅统计我作为主导师亲自指导的香港大学博士们以第一作者发表的论文。我已指导他们发表中国计算机学会 A 类（简称为 CCF A 类）论文共约 40 篇，主要包括 TPDS, SOSP, ASPLOS, MICRO, ATC, EuroSys, SIGMOD, VLDB, 以



及 IEEE S&P 等系统软件领域期刊或会议。过去 6 年，我的博士生和研究型硕士们，以第一作者和华为公司联合申请发明专利 17 项（其中 11 项专利申请属于大模型训练或 GPU 系统软件领域；3 项已获国家知识产权局授权；其余所有专利均在审查过程中，未有任何专利申请案件被驳回）。培养已毕业博士生 11 名、硕士毕业生 2 名。

1.1.1 大模型训练的科学问题

问题的严重性在千卡/万卡规模下呈现出“乘法效应”：主流系统长期受制于序列化关键路径与长尾等待，空泡会在整条流水线中累积并放大。[2] 公开分析显示，在典型配置下，数据并行同步可导致全卡超过 44% 的训练时间处于空转，平均加速卡利用率甚至可降至约 30%。[3] 这意味着：即便硬件规模继续扩张，若缺乏能够从机制上压缩关键路径的训练系统软件，训练成本、能耗与交付周期都将被长期锁定在高位，并直接制约国产千卡/万卡集群的可用性与竞争力。

均衡与计算/通信互相阻塞。本项目以并行调度与通信重排为抓手，尽可能把不可避免的同步从关键路径中移出，并将其折叠隐藏在可用的计算窗口中，从而把训练效率提升固化为可复用的系统机制与工程能力。后续第 1.2 节将围绕这一主线，给出三条代表性成果及其在真实国产千卡/万卡集群上的验证与落地。

科学假设：通用文本大模型的预训练与增强训练任务运行在通用云租户环境（如 Fat-tree 交换网络）上的千卡/万卡级加速卡集群中；模型规模与数据规模均显著超过单卡承载能力，必须采用分布式并行训练与跨设备同步。**解决的科学问题：**在上述假设下，面向文本大模型的高性能、多卡可扩展训练系统性能问题，建立可推广的并行调度与系统实现方法，并在真实国产千卡/万卡集群上完成可核对的验证与落地。

1.2 近些年已完成的代表性工作

在 §1.1.1 所述科学假设与科学问题下，本项目围绕并解决了三项关键问题（对应提纲中的 1/2/3），并在真实国产千卡/万卡集群上完成系统验证与落地：

1.2.1 “训得快”：多维并行训练中，序列化关键路径导致空泡暴露与全卡空转。 [3] 随着模型规模与训练数据规模持续增长，单卡显存与算力已无法承载主流预训练任务，业界主流转向多维并行来”把一个模型拆到很多卡上训”。[1, 10] 但多维并行也把一次迭代切成大量跨卡算子与同步原语；一旦通信与计算在关键路径上被序列化，训练就会出现”全卡空转”的系统性浪费，空泡会在整条流水线中被放大并累积。公开分析表明，主流系统普遍采用 1F1B 交织流水线（例如 2025 年 Meta 的 FlexiblePP 仍以该调度为基础），空泡可使平均加速卡利用率降至约 30%，而 DP 同步可导致全卡超过 44% 的训练时间处于空转。[2, 3]

Fold3D 揭示了“通信阻塞计算”是决定多维并行吞吐上限的关键机理，并国际首创提出段级同步调度方法 AIAO：把一次 DP 同步从”迭代末尾的整体串行阶段”重排为”按段触发、可与相邻段计算持续重叠”的同步序列，从机制上缩短落在关键路径上的等待。[3] 在理论上，段化将空泡上界推进到随有效微批次数 n （段数 $\times$ 每段微批数）按 $1/n$ 收敛的量级，形成可复用、可迁移的并行调度原语；在工程上，Fold3D 在 3072 张国产昇腾 910 的真实预训练任务中相对 Megatron 获得约 30%–40% 的端到端吞吐提升，最高可达 42.1%，把”训得快”落到可核对的系统收益上。[3]

从实际的大模型训练系统吞吐量和总训练时间这两个指标看，Fold3D 于 2024 年在 3072 张国产昇腾加速卡上完整预训练一个盘古大模型时，相比 Megatron 实现了约 30% 至 40% 的吞吐提升，并相应缩短了总训练时间 [3]。即完整预训练每

一个大模型时，Fold3D 可为我国节省约 1493 兆瓦时的主机电量。我的团队也与华为合作将 Fold3D 适配到英伟达加速卡平台，并已在 AscendSpeed 中开源 [11]。我主持的国重点项目始终坚持“两条腿走路”，即训练系统软件须同时支持国产和美国加速卡，从而形成加速卡供应的“互为备份”，降低我国被“卡脖子”的风险。

当把 Fold3D 和美国最先进的两个千卡万卡大模型多维并行训练系统软件，Megatron（英伟达发表于 SC 2021 会议）与 FlexiblePP（Meta 公司发表于 ISCA 2025 会议），进行横向对比时，我的 Fold3D 和 PipeMesh 工作在某些关键科学技术范畴具有优势 [1, 2]。PipeMesh¹²工作率先定义了“多维并行的加速卡和所有训练任务调度”的科学问题的解空间，并率先给出每个大模型训练实例的有效总算力的接近全局最优解法。在把“流水线并行的计算和数据并行的通信，进行折叠隐藏”这个科学范畴，Fold3D 相比 Megatron 与 FlexiblePP，可将关键路径上通信阻塞计算的任务数量减少约 $(N - 1)/N$ 。

在并行训练系统技术的范畴，PipeMesh 和 FlexiblePP 的科学思路相同：均以流水线并行、数据并行、张量并行这三个维度为基础，通过利用更先进的数据并行技术来进一步提升训练性能。PipeMesh 的核心多维并行调度算法已支持 ZeRO-3（即模型参数、梯度和优化器等全面参数的跨主机分布式部署）；而 FlexiblePP 的调度算法仅支持 ZeRO-2（即仅支持模型梯度和优化器的分布式部署）[2, 10, 12]。

2024 年，Fold3D 在深圳 3072 张昇腾 910B 加速卡的大型集群交付测试中，我们完成了盘古大模型预训练的实验验证。对比国外最先进的美国 Megatron 的并行调度方法，华为赞扬道：“Fold3D 调度方法和系统软件，显著提升了训练性能”（见我简历中，华为于 2026 年出具的成果转化证明书）。Fold3D 训练性能比 Megatron 的提升，及其相应的预训练盘古所需训练总时间的缩短，大约是 30%（与 Fold3D 论文的相关实验数据相符）。若 Megatron 方法在该集群预训练盘古的总需时为 90 天，则 Fold3D 缩短了 30%、即减少约 27 天。以每台“配备 16 张 910B 加速卡的主机”的功耗是大约 12 千瓦的公开资料来简单估算，3072 张卡每天能耗约为 $12\text{KW} \times 24\text{H} \times 3072/16 = 55.3$ 兆瓦时；则 27 天，仅主机的总耗电，Fold3D 就比 Megatron 节省了大约 1493.1 兆瓦时。此外，Fold3D 在其他多个难以简单估算的方面，也实现了显著的节能减碳（例如，用于保持整个集群冷却和平稳运行等方面的能耗）。

商汤公司于 2025 年也出具了表扬信（见我的简历），感谢 Fold3D（即华为 MindSpore AI 开源编程框架中的 FoldPipelineTransformer 软件）在该公司的八

千卡规模集群上进行视觉大模型高性能训练时，相比 Megatron 方法实现了显著加速。事实上，华为 MindSpore 编程框架已经开源了多种用于训练大模型的流水线并行调度实现 [4]，其中 Fold3D 方法（即 FoldPipelineTransformer）是国产自研方案，其余四种均为美国企业或大学提出的方法（例如英伟达、微软和斯坦福合研的 Megatron 方法，斯坦福的 Pipedream 方法，以及谷歌的 GPipe 方法）。

Fold3D 和 PipeMesh 主要受资助于我主持的一项国家重点研发计划项目，即科技创新 2030-“新一代人工智能国家科技重大专项”重大项目的一项支撑项目“大模型四维并行分布式训练系统”（项目代号为 2022ZD0160200）。该支撑项目直接总经费为 1.1 亿元人民币（含科技部拨款 5000 万，上海市政府拨款 5000 万，以及浦江国家实验室自筹 1000 万），聚焦千卡万卡规模的大模型并行训练调度算法的原始科学创新和系统软件产业化。

Fold3D 和 PipeMesh 的系统软件实现，已经在华为 MindSpore 开源软件库中开源，作为华为目前主力的国产千卡/万卡大模型训练系统软件（FoldPipelineTransformer） [4]。华为昇腾加速卡占国产加速卡市场 60% 以上（居主导地位），且其 AI 编程软件库（MindSpore）是全国所有集群供应商中，唯一已达到对 AI 应用 API 约 100% 覆盖率的供应商。从华为表扬信内容可见，Fold3D 软件是全国首个、也暂时是已知唯一的、达到工业级软件成熟度的通用大模型千卡/万卡训练系统软件，支持国内外著名文本大模型的高性能训练。华为表扬信还透露：（1）Fold3D 软件促进了千卡万卡集群在全国的销售落地和大范围推广，（2）Fold3D 是华为性能最好的大模型训练系统软件。

缓“卡脖”，填空白。此国重点项目在“国产千卡万卡集群上的多维并行系统软件”领域，提出和实现了一系列自主的系统新技术，有效缓解了被美国“卡脖子”的风险。华为昇腾是国产加速卡集群的先驱。2022 年，在和华为合作将 Fold3D 进行产业化过程中，我和博士生们结合昇腾集群特性，提出“多维并行计算图切图”技术：将全局计算图（即一个大模型切分成大量子切片，在所有卡上的部署和计算的数据流图）编译阶段划分为多个子图，并行下发给所有加速卡来支持跨服务器的分布式训练。我们优化了划分图策略，使其更贴合加速卡的硬件互联拓扑与计算数据流，提升了整体训练效率。

2023 年，我们进一步围绕千卡万卡训练中的跨卡通信下发与调度协同开展攻关：在成千上万张加速卡用于并行训练同一个大模型时，海量通信算子的触发时机若组织不当，容易引发跨卡通信拥塞和长尾等待，进而破坏整体训练性能的平稳性。我们提出“全局计算图与全局跨卡通信的并行融合调度”技术。此技术

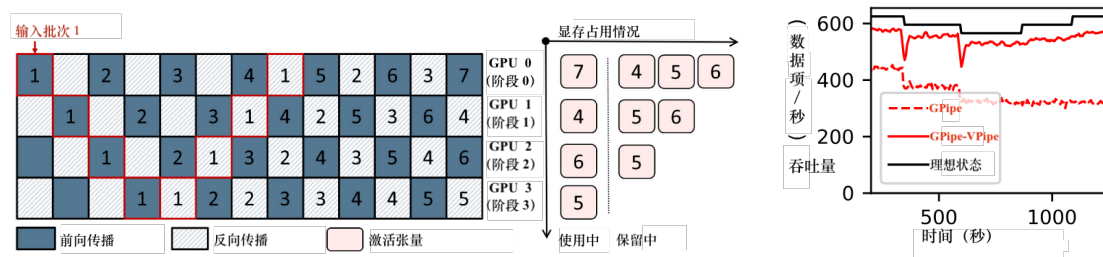


图 2: 四阶段流水线并行训练中的激活显存不均衡问题。图中展示了四个 GPU 的执行顺序, 右侧展示了各阶段的激活显存占用情况。由于激活必须到反向传播才能释放, 前段需保留更多激活副本, 显存压力远大于后段, 从而限制了训练效率与吞吐量。

图 3: 原始 GPipe 无法适应动态负载变化, 集成 vPipe 后通过在线重划分与动态显存管理, 吞吐量逼近理想值。

在提升训练时每卡平均算力利用率的同时, 还增强了大规模多维并行训练的平稳性。因此, Fold3D 软件相对于英伟达 Megatron 软件的显著性能优势 (约 30%), 得以在 2024 年运行千卡万卡、耗时数十天的盘古大模型预训练过程中, 平稳地维持。

广泛的受益面。这些独立于美国的国产软硬件协同新技术, 填补了千卡万卡规模“多维并行训练的高效率与平稳调度”系列国产技术的空白, 已使全国范围的诸多科研院所和企业显著受益, 并助力了近年的国产千卡万卡算力集群在全国广泛铺开建设的大好局面。例如, 我国的武汉、杭州、青岛和广州等十几个国家级超算中心, 以及字节跳动、DeepSeek、百度和中移动等众多科技企业, 均已采购华为升腾卡的集群或租用升腾卡的云, 用以训练或泛化微调多种大模型。Fold3D (即 FoldPipelineTransformer) 是华为升腾集群上, 目前唯一的千卡万卡训练系统软件 (包含了其“全局计算图与全局跨卡通信的并行融合调度”等关键软件技术)。因此, 当这些科研和企业单位, 运行上千张升腾卡来训练或微调其各自的大模型时 (即一个训练实例), Fold3D 这些关键软件技术就会运行起来, 显著提升了这些单位的千卡万卡级别训练实例的性能以及平稳性。

1.2.2 “训得稳”: 流水线并行训练的各加速卡阶段, 显存与计算的不平衡问题。 [7] 当一个大模型规模持续增大, 其训练实例通常需要采用流水线并行训练 (PP)。例如, 如果一个大模型拥有 80 个神经网络层 (即该模型很“长”), 难以把所有该模型的神经网络层都放到同一个加速卡内进行训练时, 流水线并行就成为支撑大模型训练的基础维度。例如, 部署 4 张加速卡作为流水线并行的 4 个阶段, 每个卡各自部署该模型的若干层神经网络 (例如, 均分, 即每个阶段各部署 20 个神经网络层)。训练数据的前向传播, 从第 0 阶段的第 0 层进入流水线, 遍历到第 3 阶段的第 79 层之后, 再进行反向传播, 把训练的计算任务依各阶段逆

向返回到第 0 层。但是，关键的科学问题是，前向和反向传播天然会造成流水线前面阶段产生的激活值，需要比后面阶段在显存中保留更长时间；而训练状态开销中，约 80% 通常来自激活值（activations）。例如，第 0 阶段需持续在显存中存储激活值，等待整个前向和反向传播在所有 4 个阶段完成后，才能逐步释放对应微批次的激活值；但是第 3（最后）阶段在完成自身阶段的前向传播并进入损失计算后，通常即可较早进行反向传播。在一个批次被切分为多个微批次并持续灌入流水线的严重情况下，第 0 阶段可能同时积累接近 4 个微批次的激活值，而第 3 阶段通常只需保留约 1 个微批次的激活值。前面阶段的激活值显存压力因此可接近后面阶段的 4 倍。这就造成流水线并行各阶段的显存使用量极易产生不平衡，从而极大影响各流水线阶段的算力利用率（例如，流水线的前面各阶段，比后面各阶段更容易出现显存占满而骤停）。

更进一步，国内外先进的视觉模型和多模态大模型多采用“超网-子网”结构进行设计和训练，以适应多种应用场景。“超网-子网”结构，是指一个大模型的所有神经网络层及其所有神经元操作符，为一个完整的集合（即一个“超网”）。在训练过程中某个时刻，如果某些层的某些操作符被激活，则此激活的层和运算符，为该集合的一个子集（即一个“子网”）。近年来，“神经网络结构搜索”等动态训练方法，即每个流水线阶段在每个前向和反向传播周期内，只会有很小比例的某些子网处于激活状态（其余所有子网则处于灭活状态）。以“超网-子网”结构设计的大模型，其子网会随着每个训练周期的推进，动态地激活/灭活，使得流水线并行的各个阶段出现计算负载和显存使用量的进一步失衡（例如，某个训练时刻，第 3 阶段率先出现显存占满而骤停）。

我的 vPipe 论文在世界上率先揭示了一个科学问题：流水线并行训练，需要“训得稳”。首先，各流水线阶段不平衡，会导致训练性能骤降（即“不稳”）。即便层数均分，各阶段的激活值和参数对显存的占用也会显著不同。常见现象是流水线并行的前面阶段先触发显存瓶颈而被迫频繁进行激活值的重算 [6]，而后面阶段却在空等，导致训练吞吐量出现骤降。[5-7] 谷歌 Gpipe 论文提出“全量重计算”方法，即前向传播时产生的激活值全量丢弃，反向传播时再重新计算和生成激活值。英伟达 Megatron 论文提出“激活函数重计算”方法。这两个方法存在两个共同的局限性：（1）本质上是消耗额外一倍的算力来缓解显存存储不足的问题，即本质上使得流水线并行训练各阶段的算力有效利用率减半；（2）两个方法皆不支持流水线并行训练的计算和显存负载动态变化的现代大模型设计方法，例如“神经结构搜索”和“超网-子网”结构（多模态大模型一般采用此设计概

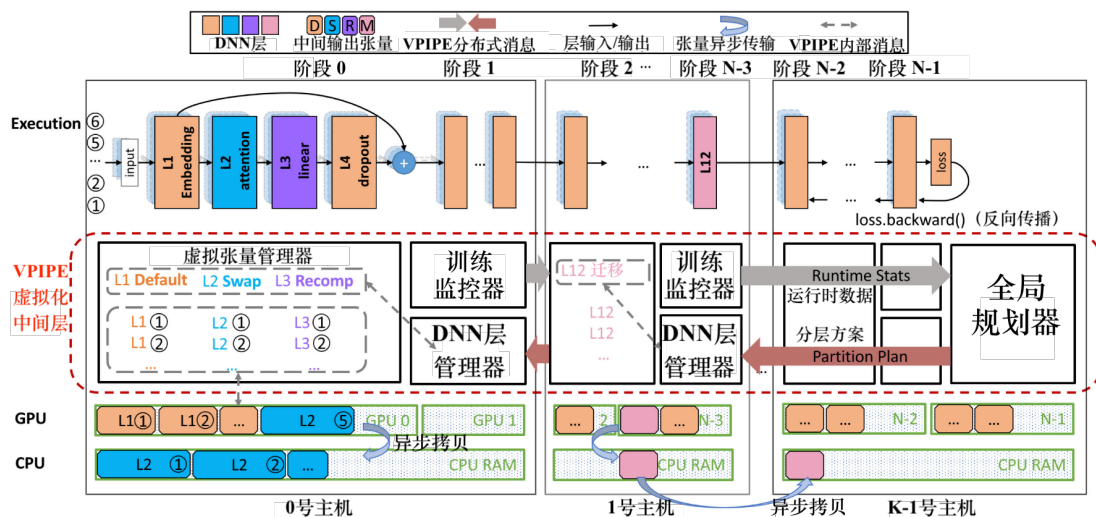


图 4: vPipe [TPDS 2022] 体系结构图: vPipe 通过监控各阶段运行状态并在线决定每层的 swap(S)/recompute(R) 与跨 GPU 的动态迁移 (M), 从而在不改变上层训练语义的前提下同时避免显存瓶颈并动态配平流水线、显著提升吞吐。

念)。例如，GPipe 使得流水线阶段有效算术单元（ALU）算力利用率低至仅约 61% [7]。

vPipe 提出”动态流水线均衡“这一新的科学方法：以“分解 + 训练时在线调整”的近似最优策略，在不暂停训练的前提下，vPipe 论文 [7]（第 4.2 章节）在世界上率先提出“选择性重计算”的训练方法：按照训练时各流水线阶段的计算负载和显存空余率，选择性地把某流水线阶段（加速卡）的显存中的某些神经网络层的激活值，置换（swap）到同主机的 CPU 内存中，并按需重计算某些神经网络层的激活值。并且，vPipe 结合跨阶段迁移（即把一个大模型的各层在各流水线阶段进行按需重划分和迁移），持续缓解计算负载和显存占用最慢的阶段，以最大化流水线并行训练的均衡性和吞吐量。vPipe 论文的实验表明，vPipe 各流水线阶段可提升到约 90% 量级；NASPipe 在超网训练中报告最高 7.8× 的加速，把“训得稳”（各流水线阶段间的计算和显存使用大致均衡），落到可实验量化的，整体训练吞吐量最大化的流水线并行的新系统软件中 [7, 13]。

vPipe 的相关科学研究，受资助于我担任主持的三项科研基金。第一，香港科研资助局 RGC 的一项面上项目（项目代号：17208223）。其次，得到了两项华为的研究和产业化基金的重点支持：（1）“面向大模型的高性能流水线并行训练系统”，并由香港大学和华为签署了科研成果转让和许可协议（项目期为 2021 年到 2023 年，合同编号：TC20210623020，项目经费 240 万港元）；以及（2）“基于加速器的数据库加速架构、理论和算法研究”（项目期为 2023 年到 2026 年，合同编号：TC20230717032，项目经费 250 万港元）。

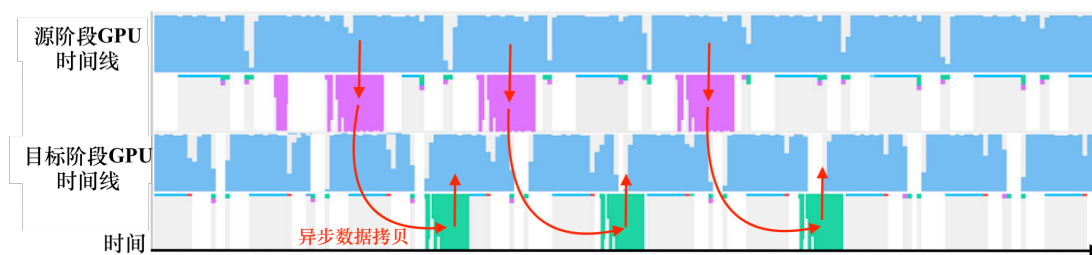


图 5: vPipe 动态迁移：在不停机的前提下把层搬到另一个 GPU，而且几乎不产生额外 GPU 空闲开销。蓝色块为 GPU 训练计算，粉块为源 GPU 把参数和反向激活从显存搬到 CPU，绿块为目标 GPU 把这些数据从 CPU 内存搬进显存。粉/绿色块和蓝色块是重叠的，异步的拷贝被藏在了正常计算的间隙里。

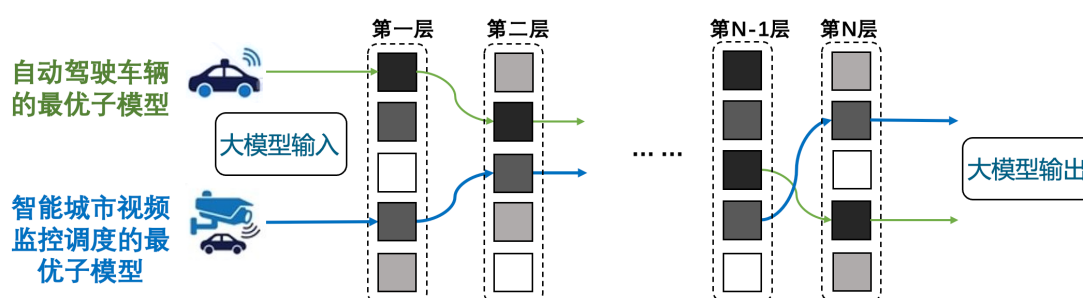


图 6: NASPipe [ASPLOS 2022] 子网概念示意图：在同一个大模型里可以激活出一个偏向“智能驾驶”的子网（绿色）和另一个偏向“交通监控”的子网（蓝色），二者共享超网中的部分层参数，但在被激活的专家（候选层）选择上不同。

在系统软件工程层面，此 vPipe 横向合作项目实现了千卡万卡规模训练的工程能力，开发了流水线高度均衡训练系统以支撑训练的软硬件工程栈。我的团队和华为围绕国产昇腾加速卡生态，以 vPipe 项目推进了软硬件协同攻关与开源生态建设；并使 vPipe 的系统软件在华为主导、支持英伟达加速卡的 MindSpeed 开源 AI 软件库 [11] 中开源。尤其是 vPipe 在世界上率先提出的“选择性重新计算”的流水线并行训练方法，已成为 MindSpeed 开源软件库中，支持流水线并行训练的三个重计算方法之一 [8]。而且，此三个方法，目前仅有 vPipe 的“选择性重计算”方法是国产自研（我们和华为联合申请了专利），其余两个方法是分别来自于美国 GPipe 和 Megatron 论文。

缓“卡脖”，填空白。过去 5 年，我与华为等企业，结合我主持的国重点项目课题二“视觉模型国产硬件适配”，推出了一系列国产化自主的软硬件协同实现。由于流水线并行训练需要始终保持各流水线阶段（加速卡）之间的显存使用量大致均衡，如前文 1.2.2 节及图 4 所示，所以主机 CPU 和同机加速卡之间会产生频繁的指令交互与数据搬移需求。例如，一个大模型训练任务生成的控制指令数量极其庞大，导致每个主机经常面临严重下发压力。我指导博士生们和华为

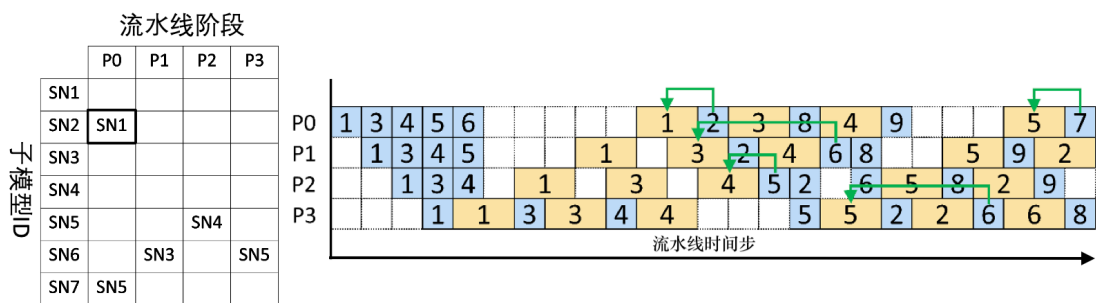


图 7: NASPipe 流水线并行训练调度示意图：蓝色块为前向读参数，黄色块为反向更新参数，块上数字为子网编号。当两个子网共享同一层参数时，编号小的黄色块必须先于编号大的蓝色块完成，即”先写后读”的因果依赖。CSP 调度机制严格保证这一顺序，同时跳过被阻塞子网去执行无冲突子网，从而减少气泡。

联合研发了“整体融合下发”机制，极大减少了主机和本机加速卡之间控制指令交互的频率。我们也研发了“异步流水调度”机制，巧妙实现了同主机内加速卡计算、跨卡通信、CPU 和加速卡之间内存拷贝等操作的“错峰”运行，显著提升了国产昇腾主机的性能。

广泛的受益面。 vPipe 的“选择性重计算”科学方法和技术，已经在 MindSpeed 软件库开源，并且是其支撑流水线并行训练的三种重计算方法中，唯一获得 MindSpeed 在官方文档中表明“推荐使用”的方法。此方法在 MindSpeed 框架中，已兼容了美国英伟达系列的加速卡硬件平台。MindSpeed 软件库目前已支持了 Transformer, GPT, Llama, 盘古, 和书生等国内外的广泛大模型的高性能训练。MindSpeed 中推荐的“选择性重新计算”的流水线并行训练方法（来源于 vPipe 论文和产业转化），不仅是 MindSpeed 三个重计算方法中唯一的国产自研方法，并且已使这些广泛的大模型进行流水线并行训练时，计算负载和显存使用量更均衡。通过 vPipe 的产业转化，以及我主持的国重点项目课题二的顺利实施，我们研发、整合和完善了一个通用的大模型训练系统（vPipe）的相关软硬件工具链，有效实现了相关 AI 工具链的国产化和国产替代。

1.2.3 “训得齐”：文本大模型增强训练中超长序列导致通信与计算难以重叠并产生显著空泡的问题，特别是当序列长度达到 32K 时专家计算仅占迭代执行时间份额约 21%。 [14] 随着预训练基座模型逐步成熟，增强训练越来越依赖“更长上下文、更强对齐”的超长序列。[9, 15, 16] 在这一趋势下，“训得齐”的关键不再是把算子堆满，而是让通信与计算在长序列条件下长期保持同频推进。

但在 MoE 等混合专家增强训练中，“训得齐”的瓶颈往往不是算力不足，而是通信与计算无法齐步前进：全互连通信（A2A dispatch/combine）与专家计算

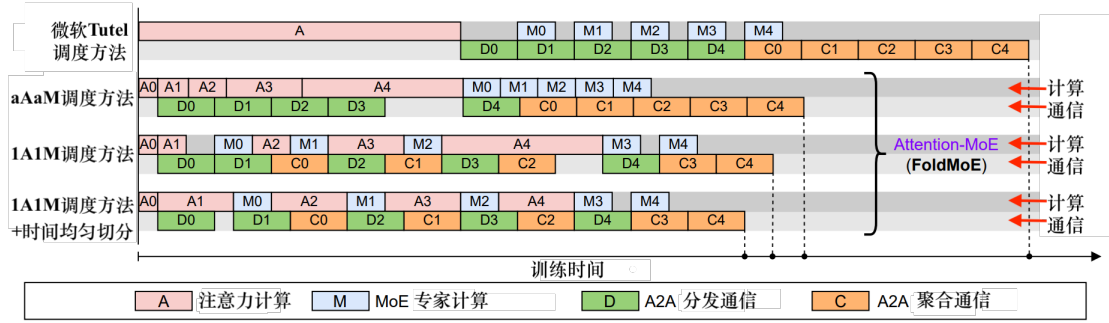


图 8: FoldMoE [ACL 2025] 提出了创新的“1A1M 归一”流水线并行调度方法，该系统训练性能显著优于微软 Tutel 局部重叠流水线调度方法，大幅减少流水线空泡，FoldMoE 已产业落地应用于 DeepSeek 并行推理（A: 注意力计算，M: 专家计算，D: 数据发送到专家，C: 把结果传回）。

相互阻塞，长尾等待在迭代内累积为空泡，导致大量时间消耗在等待而非有效计算上。结果表明，在长序列场景下，专家计算仅占迭代执行时间份额约 21%。[14] 以微软 Tutel (2023) 等静态（或仅局部重叠）的调度为代表，长序列下注意力与专家交织执行时更容易出现阶段失衡，使通信难以被稳定隐藏；更关键的是，序列长度变化会改变注意力侧时延分布，而注意力/专家（A/M）开销比的轻微波动就可能放大失衡，使得“按 token 等长切分”难以在真实工作负载中持续奏效。

“词元级”流水线并行。 FoldMoE 国际首创提出“1A1M 归一”调度这一新的并行范式：跨微批次交织注意力与专家执行、尽早触发 A2A combine，并引入 token buffer 将注意力侧的时间均匀切分与专家侧的固定粒度 micro-batching 解耦，从机制上逼近“饱和阶段通信-计算完全重叠”。在满足缓冲区供给且序列可切分为重叠度 d 的条件下，归一化空泡占比由常数开销主导并可随 d 增大被摊薄，呈“趋于零”的近似最优特性。实证上，FoldMoE 对于典型长序列比 Tutel 的端到端训练吞吐最高提升约 $2.7\times$ ，并在产业系统推理侧获得约 $1.5\times$ 的性能提升，把“训得齐”落到可规模化复用的长序列增强训练能力上。[9, 14]

上述成果也进一步明确了“词元”与“模元”（其正式定义详见 2.1）的关系：词元是模元在文本领域的典型实例；模元作为跨模态的最小并行调度单元，可自然扩展至音元、图元、语元等多模态单位，为后续统一建模、负载重整与调度优化提供一致的概念底座。

DeepSeek-V3 技术报告 [17] 与英伟达官方技术博客均指出未经优化时通信时间可占训练总时间 50% 以上，表明 FoldMoE 所解决的长序列混合专家模型通信瓶颈是国际公认的核心挑战。为解决此关键瓶颈，FoldMoE 率先提出将 Attention 层与 MoE 层解耦并进行跨微批次流水线调度的核心思想（Attention-

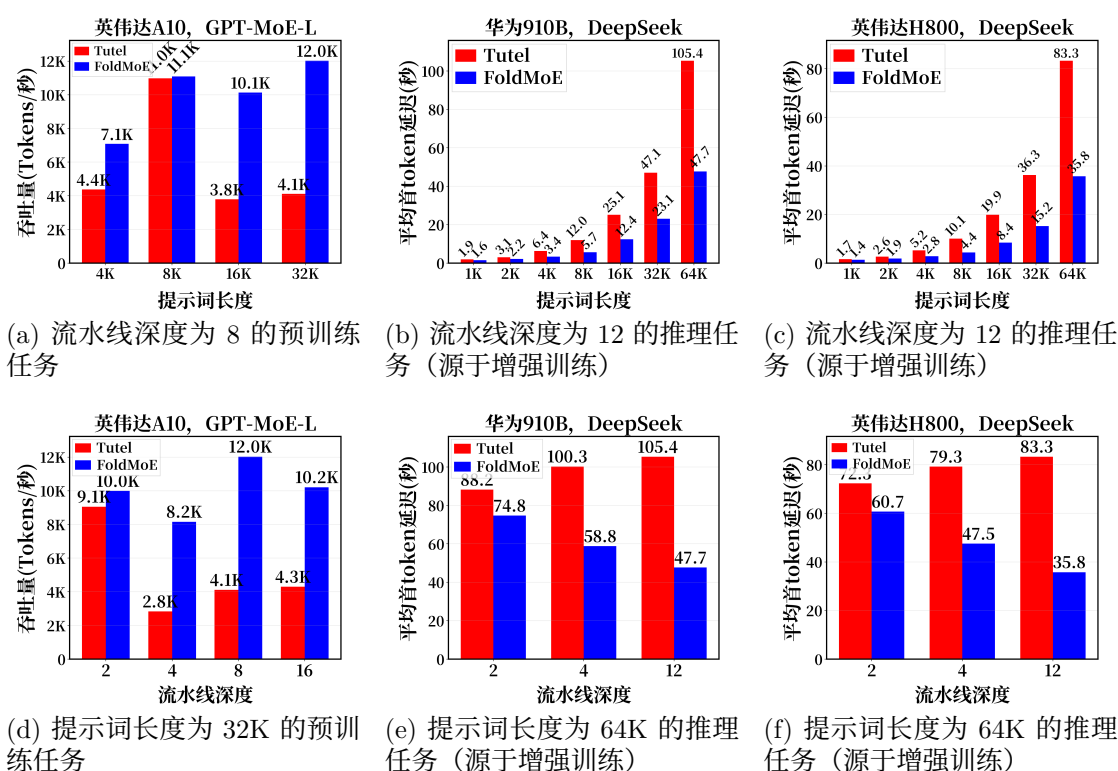


图 9: FoldMoE 在国内外著名大模型 (GPT-MoE 和 DeepSeek) 的预训练和增强训练中, 相较微软的 Tutel [9] (基于伯克利大学 vLLM 的静态调度实现), 在国产华为加速卡和国际英伟达 GPU 上显著提升吞吐并降低首 token 延时。

MoE Pipelining)。

FoldMoE 的相关研究, 主要受益于香港政府科研资助局的“科研影响力基金 (Research Impact Fund, 简称 RIF, 项目名称是 MindPipe: High-performance and Carbon-efficient Four-dimensional Parallel Training System for Large AI Models, 项目代号是 R7030-22)”的资助; 该项目直接总经费为 430 万港元, 而我是此项目的主持人。RIF 每年只资助在香港所有大学的所有理学、工程学、医学、金融和经济学、管理学以及法学等学科中, 最具有学术界或产业界影响力的一共大约 14 个科研项目。

FoldMoE 的“1A1M 归一”新算法, 已经在英伟达加速卡 and 国产昇腾加速卡上完成了系统软件实现。在英伟达加速卡集群上, 相比同类的微软 Tutel 等国际先进系统, FoldMoE 训练性能提升约 2.7 $\times$, 并保持了同等优秀的模型收敛质量。我已和华为合作, 将 FoldMoE 应用于国产 DeepSeek-V3 大模型针对“超长文本序列输入”的增强训练和推理 (服务全世界广大的终端用户), 并在昇腾加速卡上完成系统软件实现, 联合申请了一项国家专利 (见简历); 该软件在昇腾硬件上的性能, 相比国际同类的伯克利大学 vLLM 软件, 对国产 DeepSeek-V3 大模

型的增强训练和推理提升约 $1.5\times$ 。

缓“卡脖”，广出海。DeepSeek 公司的大模型的设计，已经实现了国产自主。因此，在国产加速卡上运行 DeepSeek 大模型，支撑“长文本序列的高性能增强训练和推理”应用场景，从大模型设计，科学创新原理（并行调度算法），以及软硬件技术栈方面，我国已经实现完整的自主研发。FoldMOE 已应用于支撑 DeepSeek 来理解长文本序列作为模型输入的增强训练和推理，而 FoldMOE 论文中提出的“1A1M 归一”高性能并行调度算法和软件技术栈，在 2025 年，已经实现了国产化；我们已和华为申请了专利。这些产业化努力，使得我国自研的语言类大模型及其适配全球多语种的增强训练相关国产技术栈，向海外的广泛国家和地区推广时（例如，DeepSeek 已应用于中东，用于训练阿拉伯语的大模型“ALAM”），已摆脱了对美国的依赖。

1.3 获得的国际科研奖项，青年团队，及学术评价

我作为主导师在香港大学指导博士生吴明远，获得 ACM ICSE 2025 国际会议（CCF A 类，软件工程领域）的最佳论文奖；我还指导博士生江健宇，获得 ACM ACSAC 2017 国际会议（CCF B 类，安全软件领域）的最佳论文奖。吴明远同学的论文是他在香港大学就读期间投出（该获奖论文是他的香港大学毕业论文的重要一章）；他毕业后去了南方科技大学做短暂的研究助理；通过我的全力引荐和鼓励，吴明远加入了鹏城国家实验室，担任助理研究员和进行加速卡集群的研究。关于我的其他已毕业博士生，齐冀获得了“中科院百人（B 类）”荣誉称号，并加入中科院软件所担任助理教授。沈天翔获得了国家“青年托举计划”资助。我另有至少数名已指导毕业的博士生在国外攻读博士后（例如黄东和宋浩泽），并正积极申请国内顶尖高校的教职以及“海外优青”资助。我在香港大学指导的研究助理贾薇薇，已入职美国罗德岛大学担任助理教授。我另有多名已指导毕业的博士生，例如陈旭升和赵世雄，正在华为、字节、腾讯等企业担任大模型训练系统的研发骨干。我已培养出了训练系统领域的一支朝气蓬勃的青年学者团队。

我 2016 年获得英国裘槎基金会（Croucher Foundation）的前瞻科研大奖。该奖项每年仅授予在香港所有大学的所有理学、工程学和医学的学科中，共大约三位助理教授。香港学术界普遍认为该奖项是类似于国家青年科学基金（B 类）。该奖项给予我 500 万港元的科研经费（项目期为 2016 年到 2021 年），资助形成了我在大模型训练系统领域的科研成果。

自从我 2015 年从纽约市哥伦比亚大学获得博士学位(导师是杨俊峰教授 [18]),

并旋即加入香港大学担任助理教授、博士生导师以来，我一共从科技部、香港政府和华为公司，获得了 13 个科研项目资助（我均担任项目主持人）。这 13 个项目的直接经费总额大约为 1.4 亿人民币，包括国家科技重大专项的“支撑项目”1 项（即前述的国重点项目，1.1 亿人民币），香港科研资助局的“科研影响力基金”1 项（430 万港元），香港科研资助局的“面上项目”5 项，香港创新科技局的产业化重大项目 1 项（330 万港元），华为的科研成果产业化项目 4 项（共约 780 万港元），以及英国裘槎基金会的前瞻科研大奖 1 项（500 万港元）。我主持的这所有 13 个科研项目，均为并行和分布式系统软件领域；所有已到期项目均已圆满结题。在科研成果转化成绩方面，Fold3D（大模型千卡万卡训练系统软件）实现了产业化，显著提升了使用昇腾卡集群进行训练任务的国内外广泛科研单位和企业的效益。

再例如，我的 UTEE 系列工作 [19, 20] 在世界上提出了首个，在可信安全计算处理器（例如 ARM TrustZone 和 Intel SGX）上支持高级编程语言（包含 Java 和 Python）的科学编程方法，及其安全编译工具链和运行时系统。在 UTEE 工作前（2022 年前），国内外学者和企业调用可信安全处理器时，往往只能靠低级编程语言（汇编和 C 语言）。UTEE 工作通过产业化成华为的“可信安全云服务”（TICS）的核心编程和安全运行时系统后，全世界使用该华为云服务的个人用户，云租户企业，银行，以及政府部门，均可使用高级编程语言进行安全可信处理器能力的简易、且经过完善理论证明 [19, 20] 的安全调用。

华为 2021 年和 2025 年连续向我发出两个表扬信 [21]（共两页）。其中，2025 年发出的信（第 2 页），赞扬 UTEE 的技术是世界领先，可使华为所有云用户基于高级编程语言的计算任务，有效地阻截庞大的操作系统软件中的任何潜在缺陷或安全漏洞的威胁（例如，Linux 是由全球 1500 家企业或零散组织编写的大约 4000 万行代码组成）。并且，该信透露，当每一个用 Java 编写的大数据计算函数（例如 Map 和 Reduce）在云中被调用来处理用户数据时，UTEE 系统就会被华为云的 TICS 服务调用，用来保护这些每一个计算函数的安全和机密性。

2021 年发出的信（第 1 页），透露华为云的全球 170 个运营国家中已有大约 300 万云用户使用 UTEE。另据网上多个公开新闻，从 2021 年起，中信银行和华为联合推出“华为信用卡” [22]，用 TICS（以 UTEE 作为其核心的安全保障系统）来保护该信用卡所有交易的机密性。由于“华为信用卡”由 UTEE 工作带来的高安全和机密性，广发银行 [23] 和中国银联（Union Pay） [24] 等，也已推出“华为信用卡”。目前，此华为信用卡已积累了数千万的全球用户。

并且，2022 年，UTEE 把仅在 CPU 处理器才拥有的硬件级的安全可信计算能力，通过软件级的系统创新，在世界上率先延伸到了“可安全地保护加速卡上的计算任务” [20]。由于目前世界上所有的加速卡品牌中（在欧美主要是英伟达和 AMD，在我国则有华为、海光和寒武纪等），仅有英伟达的 Hopper 或其更新型号的加速卡具备硬件级的安全可信计算能力，所以 UTEE 这个软件级安全系统延伸，对于全球的加速卡品牌均可带来显著益处。UTEE 助力促进了全世界的云计算，CPU 和 GPU 体系结构，网络空间安全，以及安全的金融和经济学等领域的进步。

尤其是国家重点项目，研发了一系列大模型训练科学新方法及其系统软件，已在浦江国家实验室，和国外最先进的大模型训练系统软件完成了量化的对比测试验证。在该国重点项目已资助我的博士们发表 CCF A 类论文 22 篇；项目结题过程中，我组织撰写了约 7.7 万字、共 494 页的《项目综合绩效自我评价报告》，其中包含 119 页的训练系统软件验收第三方测试报告。所有结题报告均已上报给科技部。

谷歌资深研究员（全球仅此两位）、美国国家工程院院士 Jeff Dean 和 Sanjay Ghemawat，和英国皇家学会会士 Michael Isard 共同在 Pathways 论文中将 vPipe 列为训练系统算力调度和加速卡资源管理的世界领先工作 [25]。ACM、IEEE 双会士 Beng Chin Ooi 与 Jian Pei 在针对表格数据的神经架构搜索研究中，将我的 NASPipe 工作列为更高效、更高性能神经架构搜索系统的代表性研究之一 [26]。欧洲科学院院士、ACM 和 IEEE 双会士 Rajkumar Buyya 教授指出 Fold3D “显著提升了并行训练性能”，并将 Fold3D 减少空泡、增大计算与通信重叠的多维并行调度算法，称为大模型训练系统的性能方面的国际最重大科学突破 [27]。字节跳动与北京大学联合团队将 Fold3D 列为大模型训练通信重叠技术的世界级代表性工作之一，称 Fold3D 技术“已成为所有大模型的多维并行训练软件中，不可或缺的方法” [28]。微软研究院评价 vPipe “通过提供更高的 GPU 利用率，比所有国际相关工作跨进了一大步” [29]。欧洲科学院院士、IEEE 会士 Alex Nicolau 在流水线并行训练论文 BPP 中，引用 vPipe 提出的双向阶段划分方法，指出该方法“允许对前向传播和反向传播进行独立划分，从而获得更均衡的阶段”，将其作为流水线阶段划分技术的代表性工作 [30]。

图灵奖得主、美国科学院院士 Yann LeCun 参与的世界模型研究团队在其论文中引用我最新工作 STAR [31] 的对抗扰动方法，指出该工作“在推理阶段通过选择性地扰动状态输入，对学习到的策略施加对抗攻击，从而增强其对环境扰动

的鲁棒性”，将其作为提升智能体在开放环境下安全可靠决策能力的代表性方法加以讨论 [32]。图灵奖得主、美国工程院院士 Michael Stonebraker 将我的 APUS 工作 [33] 列为高性能网络环境下分布式系统强一致性容错技术的代表性工作，并在讨论新型网络架构下数据库高可用系统设计时将 APUS 作为该方向的核心参考研究之一 [34]。欧洲科学院院士、ACM 和 IEEE 双会士 Torsten Hoefler 在 RDMA 通信协议综述论文中，对 APUS [33] 所采用的环形缓冲区通信机制进行了系统性分析与分类，将 APUS 列为该类高效 RDMA 数据交换协议的代表性系统实现之一 [35]。欧洲科学院院士、ACM 和 IEEE 双会士 Marc Pollefeys 在灵巧操作论文 MAPLE 中，将我的 MPI 方法（即通过预测交互来学习操作）[36] 列为通过视觉观察理解物理交互规律、从而赋予机器人灵巧操作能力的代表性方法之一，并在接触热图预测方向将其作为核心对比基线 [37]。欧洲科学院院士、IEEE 会士金耀初教授在讨论 LLM 驱动的自动化智能优化算法设计时，将我的 LLM 代码生成偏差检测与缓解研究工作 [38] 列为此方向的重要参考文献 [39]。中国科学院院士、IEEE 会士管晓宏教授在竞赛级代码生成论文 AgentConductor 中，将我的 AgentCoder [40] 列为 LLM 多智能体代码生成方向的代表性工作，并在实验中将其作为核心对比基线之一 [41]。中国工程院院士郑纬民教授与清华大学团队在并发缺陷容错领域的研究中均引用了我的 LOOM 系统工作 [42]。该团队以 LOOM 的运行时拦截机制为基础，提出了“预判不变量”方法，并将我的 LOOM 列为并发缺陷运行时容错方向的代表性工作之一，在讨论并发缺陷的修复、预防与恢复三大技术路线时将其作为核心参考文献 [43, 44]。

过去 5 年，我受邀担任了绝大部分 CCF A 级系统软件国际会议的程序委员会 (program committee) 评委。其中，我担任 OSDI, ASPLOS, ATC 和 EuroSys 的评委，均至少 3 年；我担任其的世界著名的系统软件、编译器、云计算和网络等领域的国际会议评委至少 1 年，包括 SIGCOMM, NSDI, CGO, ICDCS, Middleware 和 DSN。我担任 CCF A 类国际期刊的恒常评委，包括 TPDS, TOCS, TSE, TON, TMC 和 TDSC（这些期刊包括了系统软件，体系结构，软件工程，网络通信，移动通信，以及网络空间安全等领域）。我担任《高性能计算前沿》国际期刊的副主编。

2. 创新点

本申请的创新点面向“多模态 + 多阶段（预训练 → 增强训练）+ 多系统形态（端-云-超节点）”背景下训练系统关键假设（负载同质、通信可预测、全局同步高效等）的系统性失效，强调从既有工作中抽象出的可复用科学概念与系统

机制：将结构性不均衡导致的等待、气泡与长尾从“难以解释的工程现象”转化为可显式建模、可验证语义约束与可运行时调参的系统对象。主要创新点包括：

(1) 对前向和反向传播进行“全进全出”、重叠并行的调度方法（AIAO）。Fold3D 揭示“通信阻塞计算”是限制三维并行（3DP）吞吐上限的关键机理，并国际上率先提出 AIAO 调度方法 [3]：将原本集中在每个训练完整迭代过程的末尾的全局同步，重排为按段触发、与相邻计算持续重叠的通信序列，从科学理论上大幅减少了“数据并行通信，阻塞所有加速卡的流水线并行计算”的全局放大效应（比美国最先进的多维并行调度方法 Megatron 和 Flexible PP，减少了 $(N-1)/N$ ）。[1, 2] 该方法为后续多模态大模型下的高度均衡多维并行调度提供了研究基础。

(2) 超网/子网流水线并行与因果同步协议（CSP）。NASPipe 面向“统一主干 + 可选子网/模态模块”的超网训练形态，国际上率先提出超网/子网的系统抽象，并以 CSP 将共享参数的“先写后读”因果依赖显式化为并行约束：在保证训练可复现性的前提下跳过冲突子网、优先执行无冲突子网，从而把超网训练的并行化从经验规则提升为可验证的系统语义，并为本项目后续将“超网/子网并行”作为新的并行维度融入未来科研计划。[13]

(3) 面向长序列增强训练的近零空泡交织调度（1A1M 归一）。FoldMoE 聚焦长序列 MoE 训练中 All-to-All（A2A）通信与专家计算引发的结构性阶段失衡，国际首创提出 attention-MoE 管线化与 1A1M 归一交织调度：将注意力计算、分发/聚合与专家计算组织为可交织的阶段，并尽早触发通信以扩大可重叠窗口，从机制上逼近“通信-计算完全重叠、空泡趋于零”的可达边界。该机制支撑“长上下文 + 多模态混合输入”趋势下的增强训练效率演进，为后续在更复杂并行组合下处理长序列场景下的通信-计算时序矛盾提供可复用方法。[14]

3. 科学意义

本项目的科学意义，在于围绕大模型训练中长期制约系统上限的关键矛盾，系统揭示了负载失衡、同步长尾、通信阻塞与弱网波动等瓶颈的形成机理，并形成了从并行语义、调度原语到系统实现的连续研究链条。该链条不仅支撑千卡/万卡训练系统的高效与稳定运行，也为多模态、多阶段与端-云-超节点协同训练提供了可复用的理论基础与方法储备。

其科学意义可概括为以下六个方面：

(1) vPipe 揭示了流水线并行中显存约束与阶段失衡的运行本质。该工作将层迁移、重算与显存管理统一纳入运行时优化框架，使流水线阶段划分由静态

配置转化为可观测、可调整的系统对象，为后续多模态条件执行下的动态流水线优化奠定基础（§2.2.1 的 ModalPP 和 §2.2.2 的 UnifiedRL）。[7]

(2) NASPipe 提出因果同步并行 (CSP)，建立了超网/子网训练的并行语义。该工作将共享参数引起的“先写后读”依赖显式化为同步约束，使条件激活结构的并行执行具备可复现、可验证的系统语义，为多模态大模型“统一主干 + 可选模元子模块”的组织方式提供了可建模的并行基础（§2.2.1 的 Modal4D）。[13]

(3) Fold3D 揭示了多维并行训练的关键损失来源于通信阻塞而非单纯算力不足。该工作通过段级同步与跨段重叠重排关键路径，建立了压缩大规模同步等待与全局空转的系统方法，为千卡/万卡训练系统实现高效率与平稳运行提供了核心原理（§2.2.1 的 ModalDP）。[3]

(4) FoldMoE 提出“词元级”流水线并行，刻画了长序列增强训练的细粒度重排机制。该工作将注意力、专家计算与 All-to-All 通信纳入统一交织调度框架，显著削弱长序列场景下的结构性空泡，并为后续扩展到“模元级”流水线并行提供了直接方法参照（§2.2.1 的 ModalPP）。[14]

(5) PipeMesh 建立了调度、分片与重计算的联合优化框架。该工作表明，超大模型训练的核心问题已经不能按单一维度分别处理，而需要在吞吐、显存与通信约束之间进行全局协同编排；这一认识为更复杂的多模态和多阶段训练系统设计提供了统一优化视角（§2.2.1 的 Modal4D）。[10, 12]

总体而言，这些工作共同表明：大模型训练能力的决定性瓶颈，并不在于硬件规模本身，而在于是否掌握了能够组织异构负载、压缩同步等待并稳定并行执行的系统方法。在当前国内公开生态中，训练系统仍明显少于推理系统的背景下，这一研究链条直接支撑了我国相关能力由小规模探索走向千卡/万卡稳定训练；若缺少此类多维并行训练方法，国产大模型训练能力的成熟周期将明显延后。

进一步看，这些成果已经从文本预训练自然延伸到长序列增强训练、弱网协同训练以及“词元/模元级”统一调度框架之中。因此，其科学意义不仅在于解决既有训练系统的性能与稳定性瓶颈，更在于为本项目后续研究“多模态 + 多阶段 + 多系统形态”的统一训练系统提供了坚实的前期基础。

（二）拟开展的研究工作（建议不超过 3000 字）

着重阐述拟开展的研究工作的创新性构思，主要研究方向和初步研究方案等，请简要阐述。

2.1 研究方向与意义

近年来，多模态 (Multi Modal) 大模型正成为统一生成与理解的重要基础模

型。与只处理文本的模型相比，这类模型同时接收文本、图像、语音、视频等多种模态输入，已在流媒体视频分析、智能医疗、多模态交互生成以及具身智能等场景中显示出明确应用价值。[45–48]。为保证研究成果在国内外均可复现、可对照、可迁移，本项目将以 Janus Pro、千问（Qwen）系列、InternVL 系列、Bagel 与 DeepSeek-VL 这五个软件开源的世界著名多模态大模型，作为研究对象与实验基准。这五个大模型的设计，都已具备了某些世界领先的高智能特性，并且都包含了我国科学界和企业界的贡献 [49–53]。

这些多模态大模型相比文本大模型有三大主要区别。第一（**模型结构**），文本大模型只有一个长宽大致对称均匀的主干；虽然每个多模态大模型实现不同，但大多采用“统一主干（Backbone）+ 多模态编码子模块”的组织方式，使不同模态的数据能够进入同一套模型中联合建模与训练。第二（**模态**），文本大模型只有词元这一模态；虽然每个大模型的组成模态略有区别，但是这五个主流开源多模态大模型，是文本、图像、语音、视频这四个模态完整覆盖了这些模型公开支持的主要输入/输出形态，也代表了当前开源多模态大模型的典型设计范式，能够为全世界的后续研究提供开源、可比较的实验基础。第三（**输入和输出形态**），文本大模型通常只有文本这一种输入和输出；多模态大模型的输入输出包括多种模态，并且需要模型进行同时处理，从而“统一生成理解”。

围绕这类模型的训练，预训练与增强训练分别对应能力形成和场景适配两个关键阶段。预训练阶段需要在大规模异构数据上建立统一表征，从而形成跨模态理解与生成能力；增强训练阶段则负责把基座模型进一步对齐到具体任务与环境，使其能够适配云端持续服务的应用需求。[45, 48, 54, 55] 正因多模态模型面对的数据分布、任务结构和部署环境远比文本模型更复杂，其训练系统不仅要扩展到更多计算卡，还要在持续变化的数据组成下保持稳定和高效。

由于不同模态对应的编码开销、序列长度和计算路径并不相同，多模态大模型在多个并行维度上都极易产生加速卡计算与显存使用的不均衡问题。图 11 给出了统一输入规模下不同任务场景的计算负载差异，图 10 进一步表明，同一模型的计算量会随模态配比变化而明显偏移。当输入组成发生变化时，原本主要面向文本模型设计的静态并行策略往往难以保持稳定效率，流水线并行、数据并行和张量并行中的等待与空泡都会被放大。[1, 3] 已有系统在这一场景下也会出现明显失衡，例如图 12（a）所示的 Megatron-LM 和 Fold3D 在多模态工作负载下均存在较大的阶段不均衡与流水线气泡。

为了把这种不均衡转化为训练系统中可分析、可并行调度的元素，本项目需

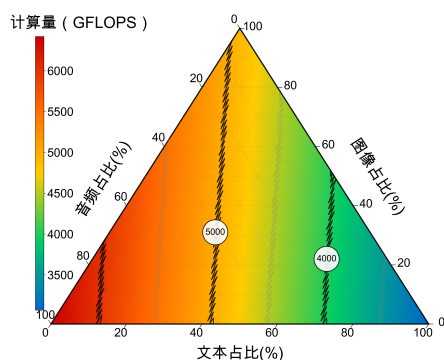


图 10: 多模态模型 (Qwen3-Omni) 计算量随输入模态占比变化而不均衡的三元图: 文、音、图的不同配比会显著改变流水线并行阶段中的计算量分布。

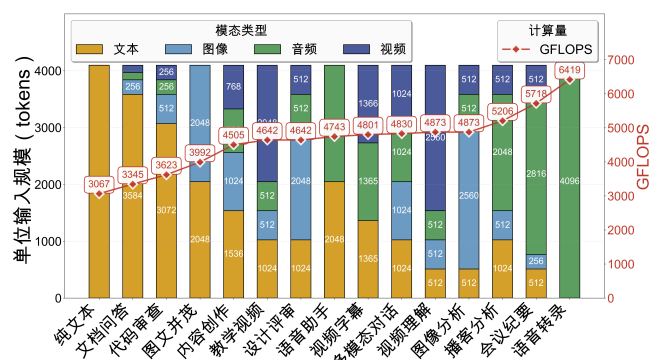


图 11: 统一输入规模 (4096 个 token) 下, 不同任务场景的多模态模型 (Qwen3-Omni) 四种模态计算负载不均衡对比图。以纯文本任务 (3067 GFLOPS) 与语音转录任务 (6419 GFLOPS) 为例, 计算量差异可达数倍。

要引入统一的调度单位。在 FoldMoE 等已有工作中, 词元 (token) 已经被证明可以作为细粒度调度单位, 用于重排长序列训练中的计算与通信过程。[14] 在多模态预训练中, 一个可调度片段往往由若干词元共同构成。基于这一认识, 本项目提出如下**科学假设**, 包含两部分。

第一, 多模态大模型训练中的主要系统瓶颈, 来自不同模态、不同模块和不同训练阶段对计算、通信与显存需求的结构性差异, 而算力集群给训练系统提供的潜在总算力其实是足够的。预训练阶段, 这种差异主要体现为编码开销、序列长度和激活路径变化带来的并行失衡; 增强训练阶段, 则进一步体现为生成、更新与在线协同在资源占用和同步节奏上的持续波动。第二, 主流多模态大模型通常由统一主干、多模态编码模块和条件计算模块构成。不同模态虽然最终进入共同训练流程, 但前序编码开销、序列长度和激活路径并不一致, 因此即使批大小相同, 不同微批次的真实训练负载仍会显著不同。

基于这一科学假设, 本申请书在国际上率先提出“模元”这一新的调度元素, 作为跨模态最小并行调度单元。一个模元可用于刻画不同模态样本在训练过程中的计算、通信与显存负载。因此, 模元可作为训练调度任务的原子元素, 用于描述、切分以及“重新平衡”不同模态输入在共同训练流程中的负载差异。后续的 ModalPP、ModalDP 和 Modal4D 将围绕预训练的并行失衡展开研究, UnifiedRL 则聚焦增强训练中的资源协同与同步稳定性问题。

2.2 初步研究方案

2.2.1 “训得快又稳”: 面向多模态大模型的高效预训练系统

对于流水线并行的一个调度元素“微批次”, 多模态大模型预训练同时面临

这个元素的数据组成不均衡和流水线执行阶段不均衡（不稳）。不同样本的文本、图像、语音、视频等模态配比持续变化，不同模态在进入统一主干前又需要经过计算强度差异显著的编码模块，因此表面上批大小相同的微批次，其真实计算量、显存占用和通信时序仍可能相差数倍，如图 11 所示。现有文本大模型的并行训练系统（如 Megatron 和 Fold3D）均建立在微批次负载近似同质的前提上，进入多模态预训练场景后，现有系统便容易出现流水线阶段失衡、气泡和训练性能显著下降，因此需要专门设计新的训练方法和系统。

围绕上述科学假设，本项目以模元为统一基础。研究内容分为流水线并行中的 ModalPP、数据并行中的 ModalDP，以及更大规模训练中的 Modal4D。三者是聚焦多模态造成的不均衡问题在不同并行维度上的创新研究和新系统设计。

多模态大模型的流水线并行训练性能，可以用空泡率统一建模。参照 Optimus 等流水线方案的效率分析 [56]，单次迭代中的空泡率可写为

$$bb = 1 - \frac{(1+c) \sum_{i=1}^N \left(\sum_{j=1}^4 m_{i,j} k_j + L_{fwd} \right)}{s (T_{LLM_base} + \Delta_{start} + \Delta_{spill})}. \quad (1)$$

其中，

$$\Delta_{start} = \frac{1}{s_{enc}} \sum_{j=1}^4 m_{1,j} k_j. \quad (2)$$

式中， N 为微批次数， s 为流水线阶段数； $m_{i,j}$ 表示第 i 个微批次中第 j 类模元的组成比例， k_j 为对应模元的单位前向计算耗时； c 为反向与前向时间比； L_{fwd} 为主干网络单批次前向耗时； T_{LLM_base} 为理想均衡情况下的主干流水线耗时； s_{enc} 为承担编码侧计算的流水线阶段数。式中的 Δ_{start} 对应“阶段启动时延”，表示编码侧重负载推迟主干稳定注入的等待； Δ_{spill} 对应“窗口外溢时延”，表示粗粒度编码计算无法嵌入零散空闲窗口而溢出到关键路径的额外时延。多模态输入一旦带来负载偏斜，这两个增量项就会迅速放大，从而直接拉高流水线气泡。例如，给定图 11 中的多模态对话任务负载，图 12 (b) 所示 Optimus 调度方法由公式 (1) 计算的空泡率为 0.6。

（一）ModalPP：“模元级”流水线并行。

当模型规模增大到单卡无法容纳完整参数和激活时，流水线并行（PP）仍是预训练扩展到更多设备的基本方法。它通常假设不同微批次的负载大致相近；但在多模态场景中，微批次负载会随模态配比和编码路径变化而明显波动。一旦局部阶段出现偏重，慢阶段便会主导关键路径，快阶段持续暴露为空泡，形成典型

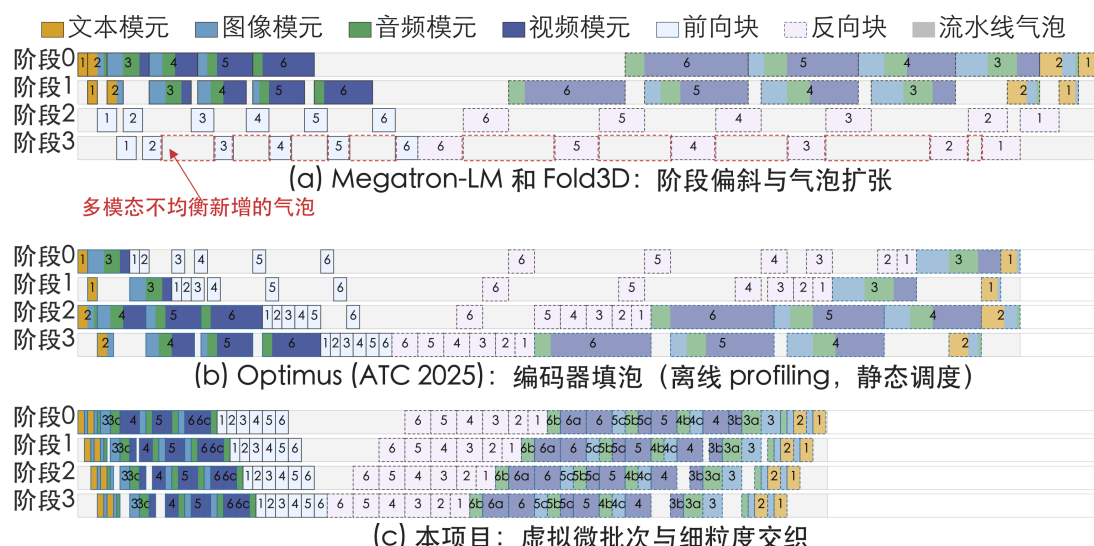


图 12: 多模态流水线并行时间表示示意。灰色区域表示流水线气泡；(a) Megatron-LM 和 Fold3D 基线在多模态输入下出现明显阶段失衡 [1, 3]；(b) Optimus 能覆盖部分空闲窗口，但依赖静态切分与静态调度 [56]；(c) ModalPP 通过模元级切分和虚拟微批次重组压缩气泡。

的阶段失衡。

例如，图 12 (a) 表明，现有流水线并行系统在多模态工作负载下会出现明显的快阶段和慢阶段。Megatron-LM 与 Fold3D 主要面向负载较均匀的训练过程设计，Optimus 虽然能够利用编码器计算覆盖部分空闲窗口，但仍依赖静态切分与静态调度；当模态配比变化时，公式 (1) 中的 Δ_{start} 和 Δ_{spill} 仍会重新累积。ModalPP 要解决的，就是负载偏重的微批次持续拉长流水线关键路径这一问题。

本项目将构建的 ModalPP 新系统，不再像现有文本大模型训练系统那样把编码器和主干绑定在同一组微批次。图 12 (c) 中，编码器和主干分别被 ModalPP 按流水线并行切分到各个阶段；其中，编码器侧再把负载偏重的混合微批次按主要模态分成更短的片段，例如 3a/3b/3c、5a/5b 和 6a/6b/6c。这样，每个编码片段的长度更接近，可以更容易填入不同阶段的空闲窗口；主干侧则仍按原生微批次的顺序推进，不必等待整个重负载编码过程全部结束。

如图 11 所示，四种模态混合输入时，原生微批次的编码开销往往明显重于纯文本场景。以图 12 (c) 中的重负载微批次 6 为例，Megatron-LM 和 Fold3D 需要等待微批次 6 的编码整体完成后，后续阶段才能接收其主干计算，因此后端阶段会持续空等。ModalPP 则把 6 拆成 6a/6b/6c 这类更短的模元片段，使前端阶段在送出 6a 后即可继续处理后续任务，而后端阶段也能尽早接收已经就绪的片段。这样，关键路径等待的对象就不再是“完整微批次 6”，而变成“已完成编码

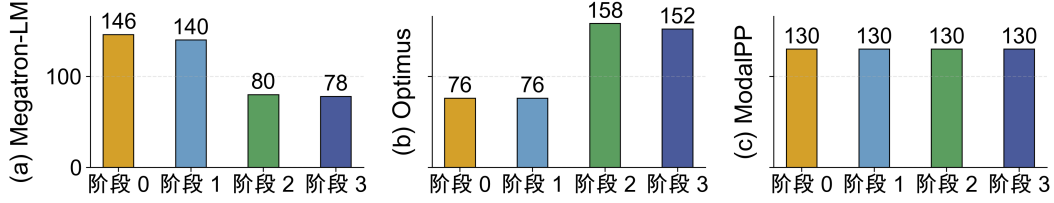


图 13: 三种方案各阶段前向计算时间的负载均衡对比。Megatron-LM 和 Optimus 的最大/最小阶段计算时间比值远大于 1，存在明显的阶段失衡；ModalPP 的各阶段计算时间最为接近，显著缓解了木桶效应。

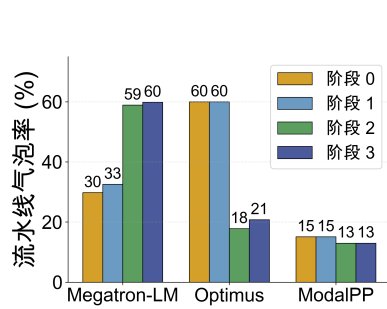


图 14: 三种流水线并行方案在多模态训练中的流水线气泡率对比，ModalPP 四阶段流水线气泡率最低。

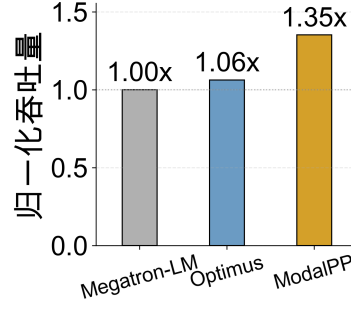


图 15: 归一化吞吐量对比：以 Megatron-LM 为基线，Optimus 提升 1.06 $\times$ ，ModalPP 提升 1.35 $\times$ 。

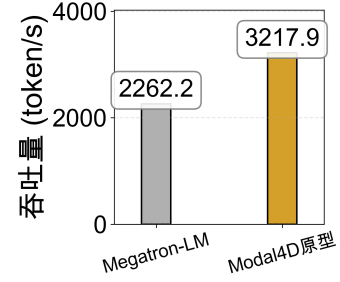


图 16: Modal4D 原型系统与 Megatron-LM 在国产腾服务器上训练 InternVL3 的吞吐对比。

的模元片段”，长等待因此被拆散。

Megatron-LM 和 Fold3D 直接以原生微批次作为流水线调度单位，一旦其中包含计算较重的图像、音频或视频模态，这些模态的编码阶段就会显著拉长整条流水线，并生成大量气泡。Optimus [ATC 2025] 虽然能够静态填补部分气泡，但仍要求编码器任务按预先性能测量的结果进行静态调度，难以在模态配比变化时继续细分和重排同一物理微批次内部的不同模态，因此 Optimus 的气泡率仍然较高。

ModalPP 提出“虚拟微批次”，用来把编码器和主干切割成的若干片段重新组装而成的新调度单位。它不是新的训练样本，而是把已经完成编码、且负载较接近的模元片段，按照主干侧的显存预算和时间窗口重新组合为一次主干注入。这样，系统不必等待整个原生微批次全部编码完成，就能先推进已经就绪的那部分工作。ModalPP 将编码器侧的细粒度切分与主干侧的流水线推进分开组织，从而形成一系列新的、计算和显存都更均衡的虚拟微批次。因此，ModalPP 的新调度算法如图 12 (c) 所示，可显著减少公式 (1) 中的 Δ_{start} 和 Δ_{spill} ，进而可使公式 (1) 计算的空泡率由 Optimus 调度方法的 0.6 压缩到 ModalPP 调度方法的约 0.2。

为使 ModalPP 的训练性能最优化，本项目将研究两个关键科学挑战。第一，模元并不是切得越细越好，哪些模元可以独立切分、切分到什么粒度仍不改变训练语义并保持结果可复现，仍需进一步刻画。第二，虚拟微批次的注入顺序会同时影响吞吐、显存占用和调度复杂度，因此还需设计显存约束下的自适应交织策略。图 13、图 14 和图 15 给出的初步结果表明，当前原型已显著改善阶段均衡性，并把训练吞吐提升到 Megatron-LM 基线的 $1.35\times$ 。

(二) ModalDP：多模态数据并行。

多模态大模型预训练数据集规模大，数据并行 (DP) 仍是关键并行维度。其主要损失并不只来自同步本身，而是来自同步前各数据并行副本（数据并行维度的各个加速卡）状态的不一致：不同副本上的模元配比、编码器负载和到达同步点的时刻并不一致，局部偏斜会在数据并行边界上放大为全局长尾。现有文本模型常用的样本级重排，在多模态场景下通常只能保证样本数接近，却难以保证各副本在编码器侧和主干侧的负载同时对齐。

本项目提出 ModalDP 系统新设计，围绕两项科学问题展开研究。第一，利用模元这一新调度元素研究副本负载重整方法，把数据并行关键计算和通信任务的“重排” (reshuffling) 从样本级调整到模元级，使不同副本在进入数据并行同步点前具有更接近的计算负载。第二，研究面向长尾副本（计算负载较大或较慢的副本）的新型细粒度同步方法。例如，我们将在 Fold3D 的细粒度（段级）同步思想基础上，进一步按“模元段已就绪”的同步点进行全规约通信 (AllReduce)，使较快副本能够更早从同步点退出，并与慢副本（尚未退出同步点）开展可重叠的计算和通信，而不再像现有系统（如 Megatron 和 Fold3D 中的数据并行方法）那样被最慢副本完全绑定。 [3]

(三) Modal4D：多维并行协同。

随着一个多模态模型的参数规模不断增大，仅靠 ModalPP（流水线并行）和 ModalDP（数据并行）这样的单一并行维度，不足以支撑超大规模多模态预训练。因此，需要融合数据并行、流水线并行和张量并行等的多维并行训练系统。基于我们的 PipeMesh 的已有基础 [12]，本项目已将 ModalPP 与 ModalDP 融入多维并行系统原型。图 16 表明，该原型在 InternVL3 训练中较 Megatron 已取得约 22.72% 的端到端吞吐提升。 [51]

但在多维并行训练时，多模态将会带来更多不均衡问题。文本模型中较容易对齐的切分边界，在多模态场景下会因为模态比例、编码计算和参数分布不同而重新失衡，因此主要面向文本大模型的 PipeMesh 系统难以直接达到最优多维并

行配置。将来，本项目将继续研究如何让 Modal4D 的上述“模元级重新平衡”方法在多个并行维度上实现最优化，不同多模态子模块如何进行跨维分配，以及数据并行的同步点方法如何与 ModalPP 流水线新调度算法进行协同优化设计，以形成最适合多模态大模型预训练的统一、通用并行训练系统。

2.2.2 “训得快又齐”：面向多模态大模型的高性能增强训练系统

当前多模态大模型已通过海量数据“预”训练，具备了跨越图、文、音、视四种模态的泛化能力，而为满足特定应用场景的适配需求，近年来，增强训练（“后”训练）也变得越来越重要。得益于多模态特性，本项目拟研究的五个已预训练的大模型（Janus Pro、Qwen 系列、InternVL 系列、Bagel 与 DeepSeek-VL），广泛覆盖了诸多高价值的实际应用场景。首先，在智能医疗领域，多模态模型被用于处理复杂的医学影像辅助诊断与多轮问诊交互 [57]。其次，在自动驾驶领域，模型通过实时处理视频流与雷达信号，进行端到端的驾驶决策与路径规划 [58]。第三，在具身智能领域，结合长音频指令与视觉环境感知的机器人能够完成长周期的复杂物理任务操作 [59]。为了让已预训练的大模型能够高效、低成本地适应上述各类异构数据集与具体业务场景，研发高性能的增强训练系统（UnifiedRL）成为当前亟待解决的关键任务。

（四）UnifiedRL：基于统一显存的多模态大模型高性能增强训练系统

多模态大模型的增强训练流程通常分为“生成”（rollout）与“训练”（Optimization）两个阶段，且生成阶段对一个模型的输入序列的长度需求，正越来越显著。在生成阶段，模型接收多模态提示词，并在推理模式下自回归地生成输出序列。随后在训练阶段，系统利用生成的序列作为训练样本，通过特定的优化算法更新模型参数。随着应用场景复杂度的提升，高分辨率图片、高帧率视频或长段音频会被编码为数量庞大的模元。这不仅极大拉长了生成序列的长度，也直接导致生成阶段的键值缓存（KV Cache）容量呈指数级增长，为底层的硬件资源管理带来了严峻考验。

科学难题。多模态大模型在生成阶段的极高显存需求与训练阶段的高算力需求之间，存在计算与显存等资源错配。图 17 对比展示了现有的两种主流部署策略及其局限性。时间共享策略（例如，微软的 DeepSpeed-Chat [60]）在同一组设备上串行执行生成与训练。这导致生成阶段受限于有限的显存带宽，无法充分发挥计算单元性能，使 GPU 长期处于低利用率状态，降低吞吐。空间共享策略（例如，字节跳动的 verl [61]）则划分出独立的设备池执行不同阶段任务。然而在长序列多模态场景下，生成节点的显存极易耗尽，引发向 CPU 内存的频繁卸载与

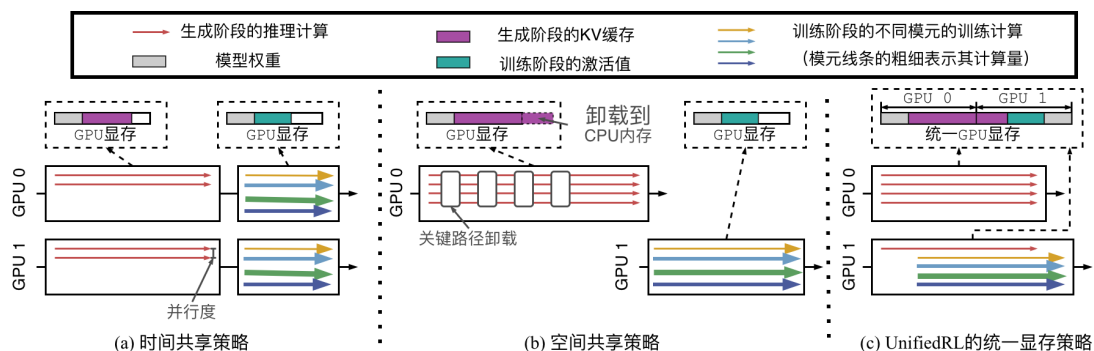


图 17: 三种部署策略对比。时间共享策略（如微软的 [60]）串行生成与训练，致 GPU 低效。空间共享策略（如字节跳动的 [61]）受显存约束触发卸载传输，生成阶段变瓶颈。我们的 UnifiedRL（统一显存策略）在推理与多训练实例间共享显存。

数据传输，进而大幅拉长了解码延迟，使得生成阶段成为瓶颈。总体而言，这两种传统策略均静态割裂了计算与显存的分配，无法自适应多模态大模型增强训练流程中两个阶段截然不同的硬件资源消耗特征。

针对上述资源错配问题，我们的 UnifiedRL 系统拟提出和解决五个核心研究点，系统性地构建面向多模态大模型的高性能和高资源使用率的增强训练系统。第一，探索跨卡的统一显存管理机制。如图 17所示，该机制通过解耦计算与显存分配，在推理实例与多任务训练实例间共享 GPU 显存，从而减少关键路径上的卸载开销并提升并行度。第二，设计自适应的“对齐”重计算策略。图 18揭示了统一显存策略带来的新挑战，即动态变化的显存会引发流水线负载失衡与气泡。该策略将提出同步各流水线阶段的重计算的新方法，有效消除气泡以保证训练流水线的连续性。第三，构建基于吞吐感知的动态显存分配算法。该算法实时监控生成与训练阶段的吞吐速度，动态微调显存边界，使两阶段速率匹配以实现全局性能最优。第四，提出“模元”感知的显存池差异化管理方法。由于不同模元的计算量与显存占用密度存在显著差异，该机制将依据四种模态的原生特征和区别，设计新型的细粒度、差异化缓存路由与调度方法。第五，探索面向多模态和极长输入序列的系统级并行优化策略。针对长序列带来的跨卡显存碎片问题，通过借鉴 FoldMoE 的长序列重排思想，设计高效的分布式键值缓存聚合与计算方法，进一步降低分布式注意力机制的延迟。

目前，我们已与华为就 UnifiedRL 展开深度合作：我和博士生们已在英伟达卡集群和华为国产卡集群中，完成了 UnifiedRL 的软件原型实现，并由我的博士生（作为第一作者）来撰写该软件的专利申请。图 19展示了 UnifiedRL 的初步实验结果，较国内外相关的最先进系统软件（包括字节跳动的 verl 软件）获得了 1.34 至 1.61 倍的整体吞吐量跃升。这些令人振奋的初步实验结果和持续深度合

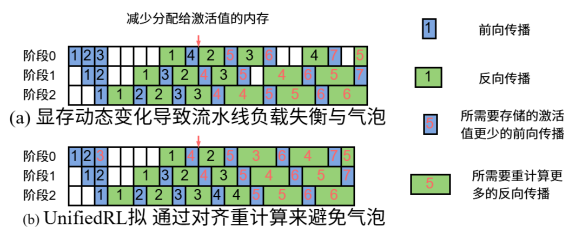


图 18: UnifiedRL 的统一显存管理策略所面临的新挑战：训练过程中显存占用动态变化，导致流水线负载失衡与气泡（图中以词元为例）；借鉴 vPipe 的卸载思想，UnifiedRL 拟通过对齐重计算来避免气泡。

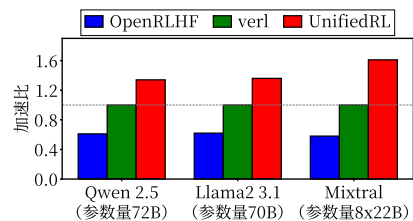


图 19: UnifiedRL 的初步实验结果：对于三种典型大模型（Qwen 和 Mixtral 是多模态大模型）比国内外著名基准系统（例如，字节跳动的 verl [61]）显著提高吞吐。

作，使 UnifiedRL 有望成为世界首个原生支持多模态大模型高性能增强训练的系统软件。

我预期 UnifiedRL 将在多模态大模型的增强训练领域达成几个重要的科学贡献。第一，本项目在计算机体系结构层面揭示并解决多模态大模型增强训练过程中生成阶段与训练阶段的资源错配问题。第二，本项目提出的跨卡统一显存管理机制，能够大幅缓解多模态长序列带来的显存碎片化瓶颈。第三，通过挖掘分布式训练的体系结构层执行策略，本项目提出的对齐重计算策略，将有效解决动态显存环境下的流水线负载失衡问题。第四，模元感知的差异化管理方法，也将为全世界加速卡集群算力资源的精细化、高性能管理等领域，提供新的科学启示和开源软件示范。

本申请书对国内外大模型的新型设计，将带来深远的科学意涵。例如，UnifiedRL 提供的跨加速卡统一显存管理的新科学方法和系统设计，从科学本质上是设想是把大模型的增强智能学习的过程所需的硬件资源，成功地“解耦化”（disaggregation，即把大模型所需计算和显存等硬件资源管理的科学边界，从“以卡为单元”的藩篱中解放出来）。多模态大模型的设计也出现了类似趋势。例如，Bagel 等大模型为了提升多模态智能并减少模态之间的干扰，正在将生成的参数集和理解参数集不断地进行解耦化设计 [53]。关于 Bagel 的生成类任务和理解类任务，若给予该模型不同的多模态输入，相应产生的这些任务对于计算和显存等硬件的负载，差异可达数倍 [53]。目前世界上的加速卡集群均以卡为单元，其计算和显存等硬件资源，难以高效地匹配这些激进的模型设计及其训练需求。因此，UnifiedRL 的“计算和显存解耦化、并统一进行管理”的思想，将有望支撑全世界的多模态大模型在未来不断地探索和实现此设计趋势。

2.3 总结

本申请书的研究计划和实验，拟同时使用国产和从国外可获取的最先进加速卡。论文研究的系统实现和实验，计划运行在百卡规模。我也将积极和世界级企业（例如华为、商汤、字节和腾讯等）紧密合作，把研究成果和某些关键实验，产业转化到千卡万卡集群之上。我计划发表 CCF A 级的训练系统或 AI 系统的论文至少 20 篇，申请相关发明专利至少 12 项。最终目标是构建一个统一、通用和开源的训练系统软件，支撑前述五个世界级的开源多模态大模型的高性能训练。

近年，我国的多模态大模型新设计不断涌现，已呈现世界领先的大好趋势；但是，美国正在对我国相关科学和技术领域，施加百般阻挠。因此，“今天”就是对本项目立项的最佳和最紧迫的历史时机。如果延后一年再考虑资助本项目，我国被“卡脖子”的问题很可能再度恶化。如果尽快资助本项目开展研究，我国将会在大模型训练领域形成一系列科学创新，并在系统软件和体系结构等技术领域继续填补国内和国际的空白。这些成果将会有效保障和促进我国的计算机、通信、半导体、AI 等学科的蓬勃发展。

（三）其他需要说明的情况

1. 申请人同年申请不同类型的国家自然科学基金项目情况（列明同年申请的其他项目的项目类型、项目名称信息，并说明与本项目之间的区别与联系；已收到自然科学基金委不予受理或不予资助决定的，无需列出）。

无

2. 具有高级专业技术职务（职称）的申请人是否存在同年申请或者参与申请国家自然科学基金项目的单位不一致的情况；如存在上述情况，列明所涉及人员的姓名，申请或参与申请的其他项目的项目类型、项目名称、单位名称、上述人员在该项目中是申请人还是参与者，并说明单位不一致原因。

无

3. 具有高级专业技术职务（职称）的申请人是否存在与正在承担的国家自然科学基金项目的单位不一致的情况；如存在上述情况，列明所涉及人员的姓名，正在承担项目的批准号、项目类型、项目名称、单位名称、起止年月，并说明单位不一致原因。

无

4. 同年以不同专业技术职务（职称）申请或参与申请科学基金项目情况（应详细说明原因）。

无

5. 其他。

无

参考文献

- [1] Narayanan D, Shoeybi M, Casper J, et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM[C/OL] // Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21). St. Louis, MO, USA: ACM, 2021[2026-02-08]. <https://arxiv.org/abs/2104.04473>.
- [2] Chu W, Xie X, Yu J, et al. Scaling Llama 3 Training with Efficient Parallelism Strategies[C/OL] // Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25). Tokyo, Japan: ACM, 2025[2026-03-11]. <https://doi.org/10.1145/3695053.3731410>.
- [3] Li F, Zhao S, Qing Y, et al. Fold3D: Rethinking and Parallelizing Computational and Communicational Tasks in the Training of Large DNN Models[J]. IEEE Transactions on Parallel and Distributed Systems, 2023[2026-02-09].
- [4] MindSpore Team. MindSpore Pipeline Transformer[J/OL]. GitHub repository, 2026[2026-03-15]. https://github.com/mindspore-ai/mindspore/tree/master/mindspore/ccsrc/frontend/parallel/pipeline_transformer.
- [5] Narayanan D, Harlap A, Phanishayee A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training[C/OL] // Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19). 2019[2026-02-09]. <https://www.microsoft.com/en-us/research/wp-content/uploads/2019/08/pipedream.pdf>.
- [6] Huang Y, Cheng Y, Bapna A, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism[J/OL]. arXiv preprint arXiv:1811.06965, 2018[2026-02-09]. <https://arxiv.org/abs/1811.06965>.
- [7] Zhao S, Li F, Chen X, et al. vPipe: A Virtualized Acceleration System for Achieving Efficient and Scalable Pipeline Parallel DNN Training[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2021[2026-02-09]. <http://dx.doi.org/10.1109/TPDS.2021.3094364>.
- [8] MindSpeed Team. MindSpeed-LLM Recompute Feature Documentation[J/OL]. Gitee repository documentation, 2026[2026-03-16]. https://gitee.com/ascend/MindSpeed-LLM/blob/2.1.0/docs/pytorch/features/recompute_relative.md.
- [9] Hwang C, Cui W, Xiong Y, et al. Tutel: Adaptive Mixture-of-Experts at Scale[J]. arXiv preprint arXiv:2206.03382, 2023.
- [10] Rajbhandari S, Rasley J, Ruwase O, et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models[C/OL] // Proceedings of the International Conference for

- High Performance Computing, Networking, Storage and Analysis (SC '20). 2020[2026-02-09]. <https://arxiv.org/abs/1910.02054>.
- [11] Ascend Team. AscendSpeed[J/OL]. GitHub repository, 2026[2026-03-15]. <https://github.com/Ascend/AscendSpeed>.
- [12] Li F, Zhao S, Qing Y, et al. PipeMesh: Achieving Memory-Efficient Computation-Communication Overlap for Training Large Language Models[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2025[2026-03-01]. <http://dx.doi.org/10.1109/TPDS.2025.3583983>.
- [13] Zhao S, Shen T, Li F, et al. NASPipe: High Performance and Reproducible Pipeline Parallel Supernet Training via Causal Synchronous Parallelism[C/OL] // Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22). 2022[2026-02-09]. <http://dx.doi.org/10.1145/3503222.3507735>.
- [14] Zhu G, Lei L, Qing Y, et al. FoldMoE: Efficient Long Sequence MoE Training via Attention-MoE Pipelining[C/OL] // Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL 2025). 2025[2026-02-09]. <https://arxiv.org/abs/2502.20549>.
- [15] Dao T, Fu D Y, Ermon S, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness[C/OL] // Advances in Neural Information Processing Systems (NeurIPS). 2022[2026-02-10]. <https://arxiv.org/abs/2205.14135>.
- [16] Dao T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning[C/OL] // International Conference on Learning Representations (ICLR). 2024[2026-02-10]. <https://arxiv.org/abs/2307.08691>.
- [17] DeepSeek-AI. DeepSeek-V3 Technical Report[J/OL]. arXiv preprint arXiv:2412.19437, 2024[2026-03-15]. <https://arxiv.org/abs/2412.19437>.
- [18] Yang J. Junfeng Yang's Homepage[H/OL]. [2026-03-17]. <https://www.cs.columbia.edu/~junfeng/>.
- [19] Jiang J, Chen X, Li T O, et al. Uranus: Simple, Efficient SGX Programming and its Applications[C/OL] // Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (ASIA CCS '20). Taipei, Taiwan: ACM, 2020[2026-03-16]. <https://www.cs.hku.hk/~heming/papers/asiaccs20-uranus.pdf>.
- [20] Jiang J, Qi J, Shen T, et al. CRONUS: Fault-isolated, Secure and High-performance Heterogeneous Computing for Trusted Execution Environment[C/OL] // 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO). Chicago, IL, USA: IEEE, 2022: 124–143[2026-03-16]. <https://doi.org/10.1109/MICRO56248.2022.00019>.
- [21] Huawei. Huawei Acknowledgement Letters for UTEE and TICS[J/OL]. Acknowledgement letter, 2025[2026-03-16]. <https://hemingcui.github.io/doc/utee-ack.pdf>.

- [22] Huawei Cloud. 中信银行与华为联合推出华为信用卡 [J/OL]. 新闻稿, 2021[2026-03-16]. <https://www.huaweicloud.com/intl/zh-cn/news/2020/20210604100805740.html>.
- [23] 南方都市报. 广发银行推出华为信用卡相关新闻 [J/OL]. 新闻报道, 2024[2026-03-16]. <https://m.mp.oooo.com/a/BAAFRD0000202412111033651.html>.
- [24] 百家号. 中国银联相关华为信用卡新闻 [J/OL]. 新闻报道, 2020[2026-03-16]. <https://baijiahao.baidu.com/s?id=1663432459663052187&wfr=spider&for=pc>.
- [25] Barham P, Chowdhery A, Dean J, et al. Pathways: Asynchronous Distributed Dataflow for ML[C/OL] // Proceedings of Machine Learning and Systems (MLSys 2022). 2022[2026-03-16]. <https://arxiv.org/abs/2203.12533>.
- [26] Xing N, Cai S, Luo Z, et al. Anytime Neural Architecture Search on Tabular Data[J/OL]. arXiv preprint arXiv:2403.10318, 2024[2026-03-16]. <https://arxiv.org/abs/2403.10318>.
- [27] Zhou G, Tian W, Buyya R, et al. UMPIPE: Unequal Microbatches-Based Pipeline Parallelism for Deep Neural Network Training[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2025[2026-03-16]. <https://doi.org/10.1109/TPDS.2024.3515804>.
- [28] Chang L-W, Bao W, Hou Q, et al. FLUX: Fast Software-based Communication Overlap On GPUs Through Kernel Fusion[J/OL]. arXiv preprint arXiv:2406.06858, 2024[2026-03-16]. <https://arxiv.org/abs/2406.06858>.
- [29] Jangda A, Yadav A, Jacobs S, et al. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads[C/OL] // Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22). 2022[2026-03-16]. <https://doi.org/10.48550/arXiv.2105.05720>.
- [30] Cui Y, Liu C, Xiao Z, et al. BPP: Branch Pipeline Parallelism Strategy for Accelerating DNN Training[J/OL]. Neurocomputing, 2026[2026-03-16]. <https://doi.org/10.1016/j.neucom.2026.133312>.
- [31] Zhang Z, Duan T, Lin Z, et al. State-Aware Perturbation Optimization for Robust Deep Reinforcement Learning[J/OL]. arXiv preprint arXiv:2503.20613, 2025[2026-03-16]. <https://arxiv.org/abs/2503.20613>.
- [32] Parthasarathy A, Kalra N, Agrawal R, et al. Closing the Train-Test Gap in World Models for Gradient-Based Planning[J/OL]. arXiv preprint arXiv:2512.09929, 2025[2026-03-16]. <https://arxiv.org/abs/2512.09929>.
- [33] Wang C, Jiang J, Chen X, et al. APUS: Fast and Scalable Paxos on RDMA[C/OL] // Proceedings of the ACM Symposium on Cloud Computing (SoCC '17). 2017[2026-03-16]. <http://dx.doi.org/10.1145/3127479.3128609>.
- [34] Zamanian E, Yu X, Stonebraker M, et al. Rethinking Database High Availability with RDMA Networks[J/OL]. Proceedings of the VLDB Endowment, 2019, 12(11): 1637–1650[2026-03-16]. <https://doi.org/10.14778/3342263.3342639>.

- [35] Taranov K, Fischer F, Hoefler T. Efficient RDMA Communication Protocols[J/OL]. arXiv preprint arXiv:2212.09134, 2022[2026-03-16]. <https://arxiv.org/abs/2212.09134>.
- [36] Zeng J, Bu Q, Wang B, et al. Learning Manipulation by Predicting Interaction[C/OL] // Proceedings of Robotics: Science and Systems (RSS 2024). 2024[2026-03-16]. <https://arxiv.org/abs/2406.00439>.
- [37] Gavryushin A, Wang X, Malate R J S, et al. MAPLE: Encoding Dexterous Robotic Manipulation Priors Learned From Egocentric Videos[J/OL]. arXiv preprint arXiv:2504.06084, 2025[2026-03-16]. <https://arxiv.org/abs/2504.06084>.
- [38] Huang D, Zhang J M, Bu Q, et al. Bias Testing and Mitigation in LLM-based Code Generation[J/OL]. ACM Transactions on Software Engineering and Methodology, 2025[2026-03-16]. <https://doi.org/10.1145/3724117>.
- [39] Zheng Y, Zhang L, Li K, et al. A Survey on Large Language Models Driven Meta-Optimizers for Automated Intelligent Optimization[J/OL]. Artificial Intelligence Review, 2026, 59[2026-03-16]. <https://doi.org/10.1007/s10462-025-11470-w>.
- [40] Huang D, Zhang J M, Bu Q, et al. AgentCoder: Multi-Agent Code Generation with Effective Testing and Self-optimisation[J/OL]. arXiv preprint arXiv:2312.13010, 2023[2026-03-16]. <https://arxiv.org/abs/2312.13010>.
- [41] Wang S, Lu R, Yang Z, et al. AgentConductor: Topology Evolution for Multi-Agent Competition-Level Code Generation[J/OL]. arXiv preprint arXiv:2602.17100, 2026[2026-03-16]. <https://arxiv.org/abs/2602.17100>.
- [42] Wu J, Cui H, Yang J. Bypassing Races in Live Applications with Execution Filters[C/OL] // 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10). 2010[2026-03-16]. <https://www.usenix.org/conference/osdi10/bypassing-races-live-applications-execution-filters>.
- [43] Zhang M, Lu S, Qi S, et al. AI: A Lightweight System for Tolerating Concurrency Bugs[C/OL] // Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014). 2014[2026-03-16]. <https://doi.org/10.1145/2635868.2635885>.
- [44] Deng D, Jin G, de Kruijf M, et al. Fixing, Preventing, and Recovering from Concurrency Bugs[J/OL]. Science China Information Sciences, 2015, 58 : 052105[2026-03-16]. <https://doi.org/10.1007/s11432-015-5315-9>.
- [45] Lin B, Bin Y, Ye Y, et al. Video-LLaVA: Learning United Visual Representation by Alignment Before Projection[J]. arXiv preprint arXiv:2311.10122, 2023.
- [46] Tu T, Azizi S, Driess D, et al. Towards Generalist Biomedical AI[J]. arXiv preprint arXiv:2307.14334, 2023.
- [47] Bubeck S, Chandrasekaran V, Eldan R, et al. Sparks of artificial general intelligence: Early experiments with GPT-4[J]. arXiv preprint arXiv:2303.12712, 2023.

- [48] Driess D, Xia F, Sajjadi M S, et al. PaLM-E: An Embodied Multimodal Language Model[C] // International Conference on Machine Learning (ICML). 2023 : 8469–8488.
- [49] Chen X, Wu Z, Liu X, et al. Janus-Pro: Unified Multimodal Understanding and Generation with Data and Model Scaling[J/OL]. arXiv preprint arXiv:2501.17811, 2025[2026-03-15]. <https://arxiv.org/abs/2501.17811>.
- [50] Xu J, Guo Z, Hu H, et al. Qwen3-Omni Technical Report[J/OL]. arXiv preprint arXiv:2509.17765, 2025[2026-03-15]. <https://arxiv.org/abs/2509.17765>.
- [51] Zhu J, Wang W, Chen Z, et al. InternVL3: Exploring Advanced Training and Test-Time Recipes for Open-Source Multimodal Models[J/OL]. arXiv preprint arXiv:2504.10479, 2025[2026-03-15]. <https://arxiv.org/abs/2504.10479>.
- [52] Wu Z, Chen X, Pan Z, et al. DeepSeek-VL2: Mixture-of-Experts Vision-Language Models for Advanced Multimodal Understanding[J/OL]. arXiv preprint arXiv:2412.10302, 2024[2026-03-15]. <https://arxiv.org/abs/2412.10302>.
- [53] ByteDance Seed Team. BAGEL: Unified Model for Multimodal Understanding and Generation[J/OL]. GitHub repository, 2025[2026-03-15]. <https://github.com/ByteDance-Seed/Bagel>.
- [54] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[C] // Advances in Neural Information Processing Systems (NeurIPS) : Vol 35. 2022 : 27730–27744.
- [55] Cui C, Ma Y, Cao X, et al. Drive as You Speak: Enabling Human-Like Interaction with Large Language Models in Autonomous Vehicles[J]. IEEE Transactions on Intelligent Vehicles, 2024.
- [56] Feng W, Chen Y, Wang S, et al. Optimus: Accelerating Large-Scale Multi-Modal LLM Training by Bubble Exploitation[C/OL] // Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC '25). 2025 : 161–177[2026-03-05]. <https://www.usenix.org/conference/atc25/presentation/feng>.
- [57] Tu T, Azizi S, Driess D, et al. Towards generalist biomedical AI[J]. arXiv preprint arXiv:2404.18416, 2024.
- [58] Wen L, Yang X, Fu D, et al. On the road with GPT-4V(ision): Early explorations of visual-language model on autonomous driving[J]. arXiv preprint arXiv:2311.05332, 2023.
- [59] Brohan A, Brown N, Carbajal J, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control[C] // Conference on Robot Learning (CoRL). 2023 : 328–343.
- [60] Yao Z, Aminabadi R Y, Ruwase O, et al. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales[J]. arXiv preprint arXiv:2308.01320, 2023.
- [61] Sheng G, Zhang C, Ye Z, et al. Hybridflow: A flexible and efficient rlhf framework[J]. arXiv preprint arXiv:2409.19256, 2024.